Memory Organization

Dr. Vegesna S M Srinivasavarma

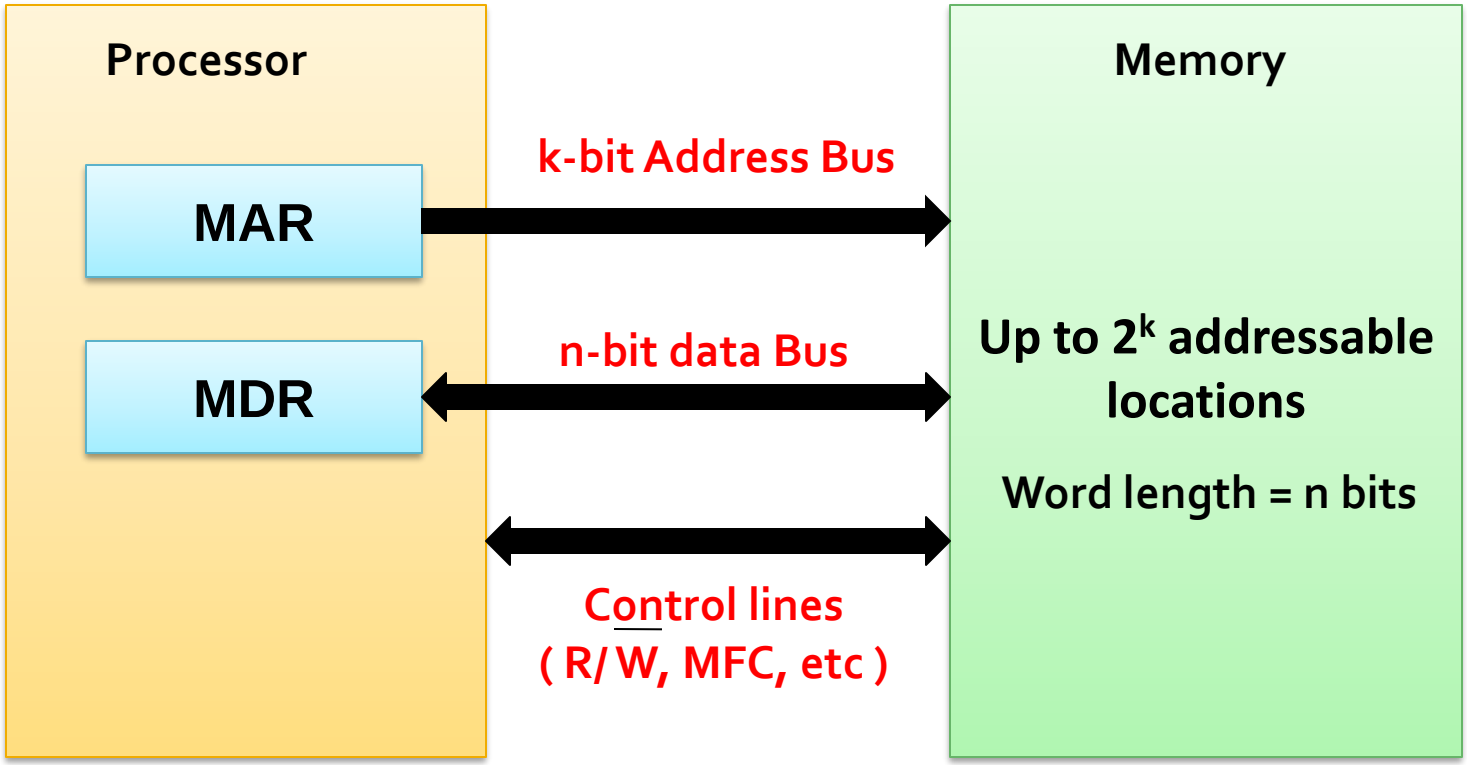
Outline

- Memory Cells – SRAM & DRAM
- Internal Organization of Memory Chip
- Memory Hierarchy
- Cache Memory organization
- Virtual memory system design

Memory

- It’s all about storing bits – binary digits
- Vacuum tubes, CRT, Drums, Disks and ICs
- Issues – size, cost and speed
- Semiconductor memories (Chips)

Basic Concepts



Connection of Memory to the Processor

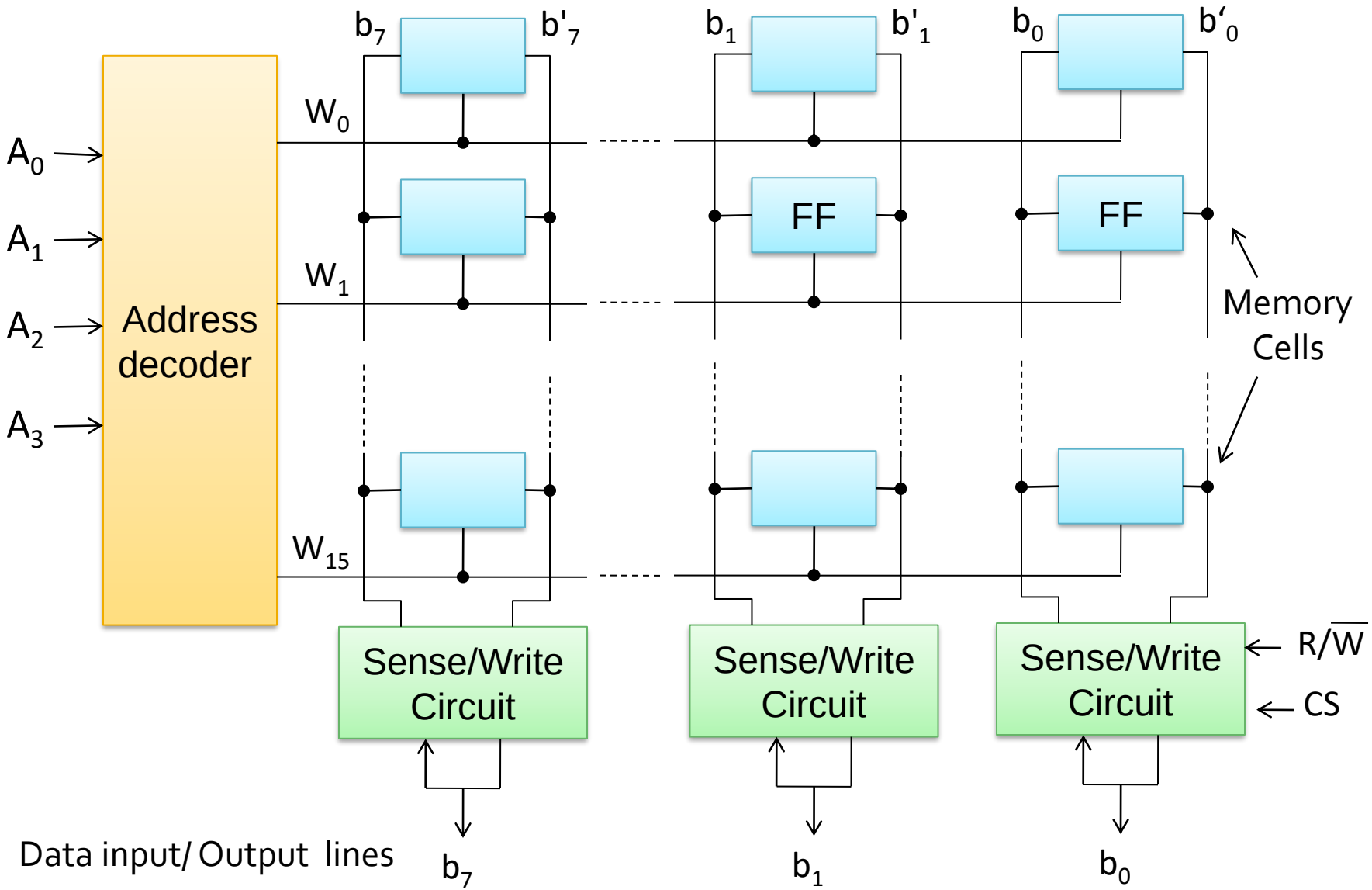
Terminology

- *Memory Access Time*: Time between read and the MFC (Memory Function Complete) signal.
- *Memory Cycle Time*: The minimum time delay required between the initiation of two successive memory operations.
- *Random Access Memory (RAM)*: Any location can be accessed for a read or write operation in some fixed amount of time that is independent of the location's address.

Semiconductor RAM Memories

- Semiconductor memories are available in a wide range of speeds.
- Their cycle time range from 100ns to less than 10ns.
- Obviously the speed of the processor depends on the speed of the memory

Internal Organization of Memory Cells

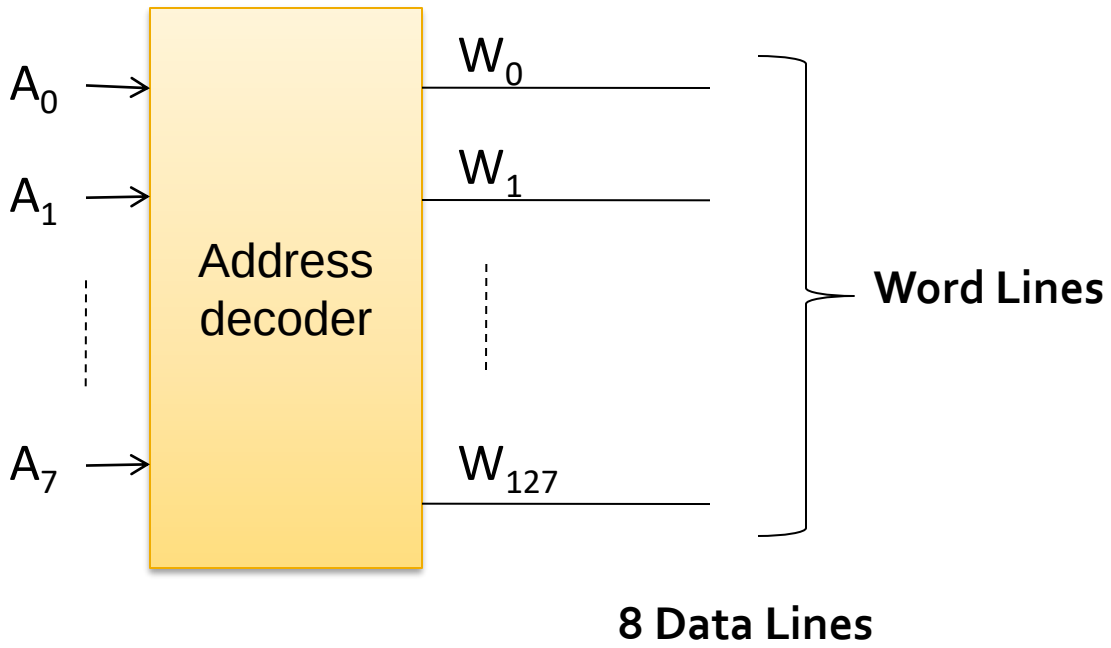


Internal Organization of Memory Cells

- Above Example: 128 bit chip
  - It have 4 address lines and 8 data lines
  - Using 4 address lines ($A_3A_2A_1A_0$) 16 word signals (W_{15}…W_1W_0) are generated with help of decoder 4:16 decoder
  - 16 words of each 8 bits (= 16 x 8 bits = 16 bytes)
  - For any given input address lines ($A_3A_2A_1A_0$) only one of the word lines from W_{15} … W_0 will be active at a time.
  - Which in turn enable the complete row of the memory cells connected to W_i bit
  - Using R/$\overline{W}$ and CS signal one can enable the Sense/Write Circuits of the all bits ranging from b_7 to b_0

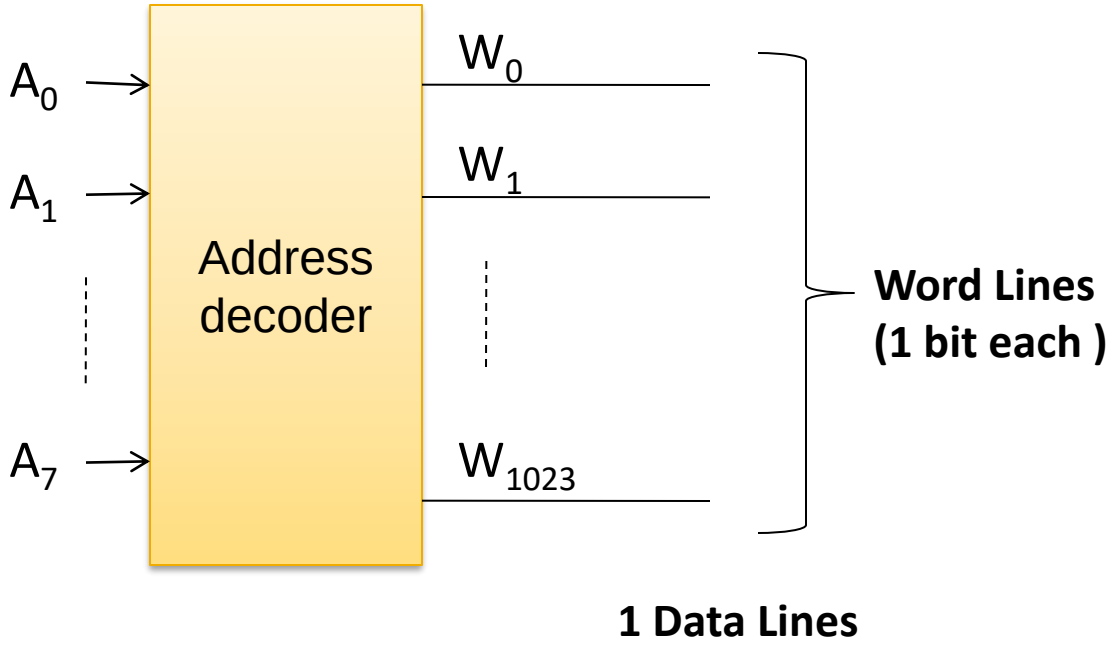
Chip with 1024 Memory Cells

- 1024 bits can be planned as 128 x 8 memory
- Total address lines required $128 = 2^N \rightarrow N = \log_2 128 = 7$, it means it should have $A_7 .. A_1 A_0$
- It mean 128 x 8 bits (Similar to 16 x 8 bits)

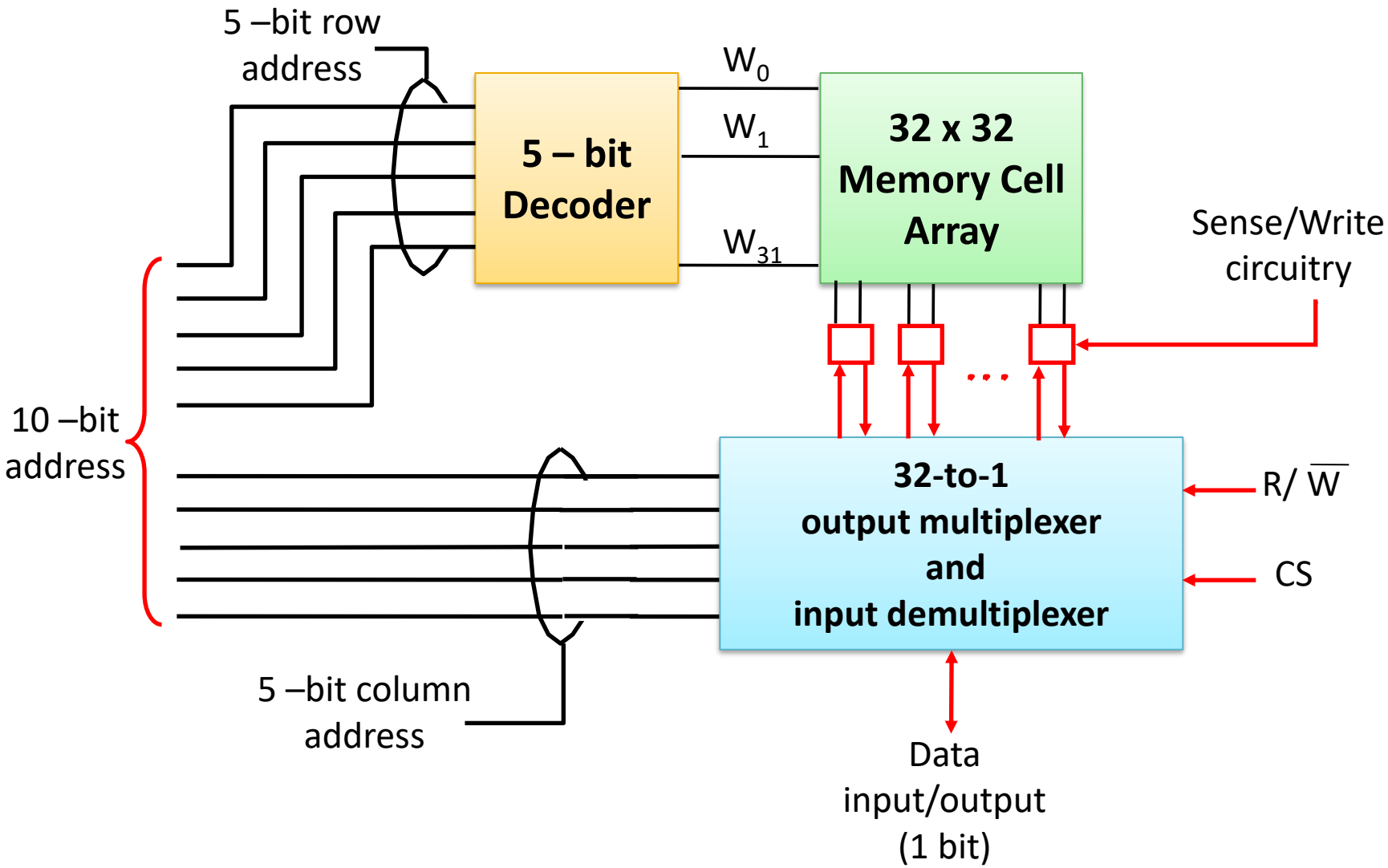


Chip with 1024 Memory Cells

- Or it could be 1024 x 1



Organization of a 1K × 1 memory chip

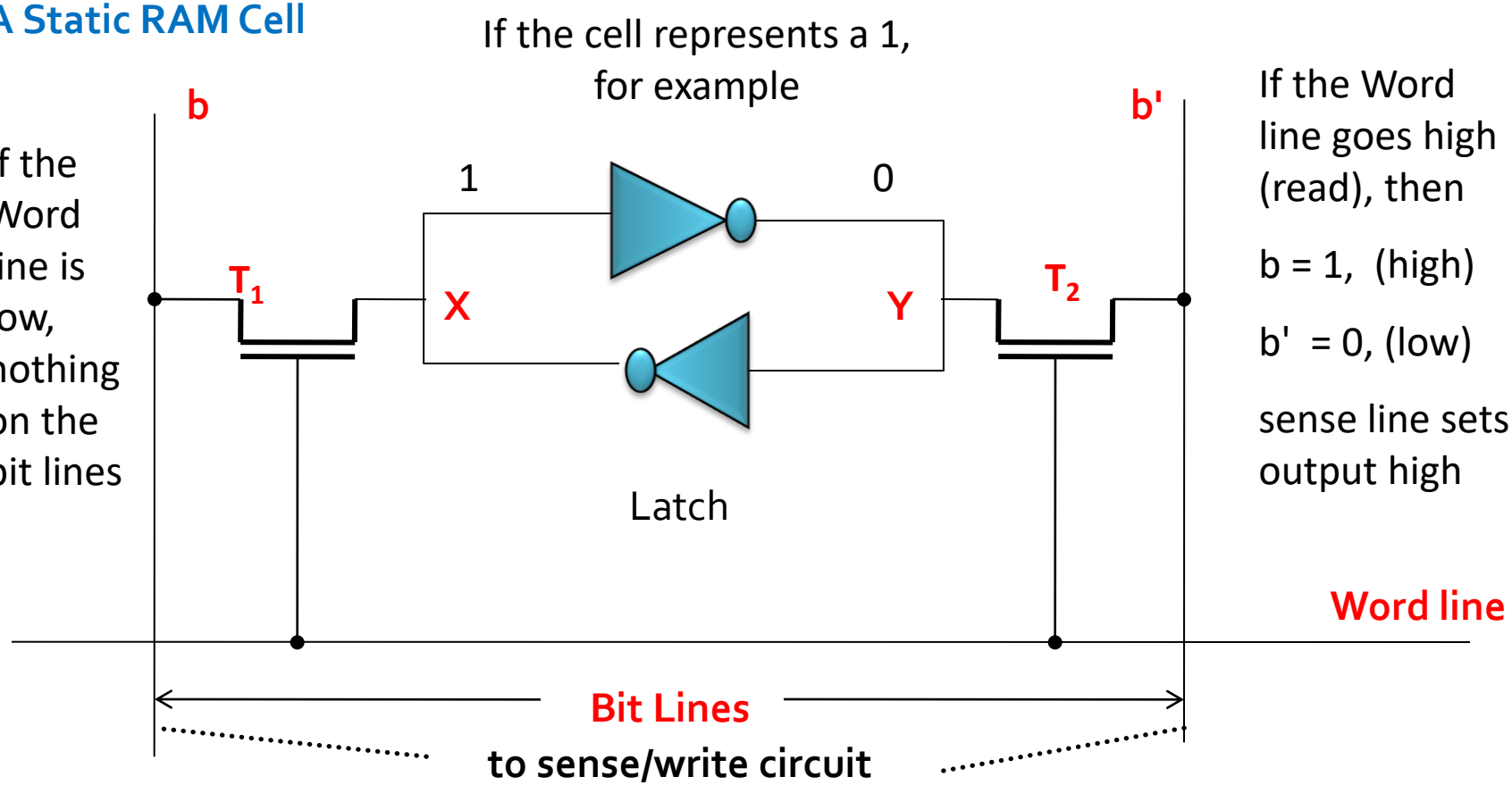


Static Memory

- Static: retains its state (content) as long as power is applied
  - Data will be erased from the cell if powered off (volatile)
- SRAM: Static RAM
  - Fast
  - Expensive

Semiconductor RAM Memories

A Static RAM Cell



- If the Word line goes high (read) ***b = 1, b' = 0***
- To write (say 0), put **b = 0**, **b' = 1**, set Word line high, latch changes

SRAM Cells

- Two inverters are cross connected to form a latch
- The latch is connected to two bit lines by transistors T_1 and T_2.
- These transistors acts as switches that can be opened or closed under the control of word line
- When word line at ground level, the transistors are turned off and the latch retains its state
- Example:
  - Let us assume the cell is in state 1 if the logic value at point X is 1 and at point Y is 0.
  - This state is maintained as long as the signal on the word line is at ground level

SRAM Cells

- Read operation
  - The word line is activated to close the switches T1 and T2
  - If the cell in state 1, the signal on bit line b is high and the signal on bit line b’ is low
  - The opposite is true if the cell is in state 0.
  - Sense/write circuits at the end of the bit lines monitor the state of b and b’ and set the output accordingly
- Write operation
  - The state of the cell is set by placing the appropriate value on bit line b and its complement on b’.
    - And then activating the word line
  - This forces the cell into the corresponding state.
  - The required signals on the bit lines are generated by the sense/write circuit.

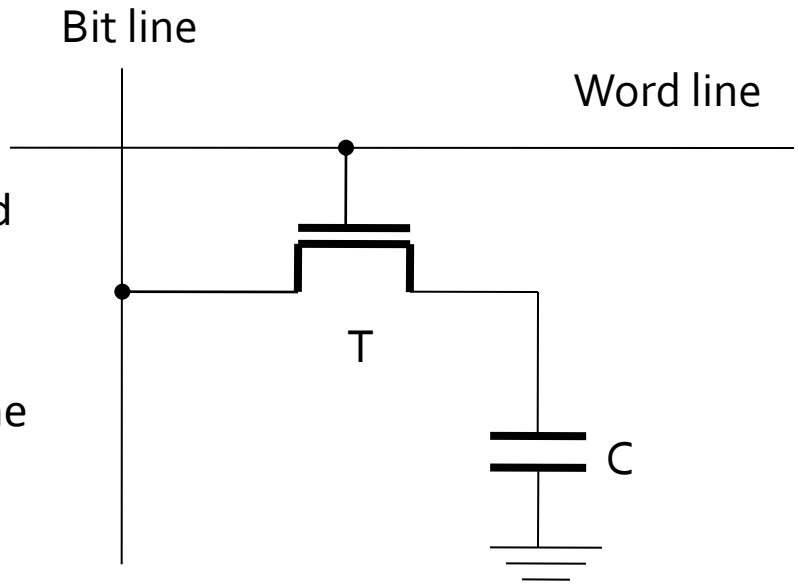
Dynamic RAM (DRAM)

- Simple cells, higher density
- 1 million to 16 million bits or more per chip
- Less expensive
- But DRAM cells do not retain their state
- Must be refreshed periodically

DRAM Cell

capacitor is charged to write a 1

(voltage applied to Word line and to the Bit line)

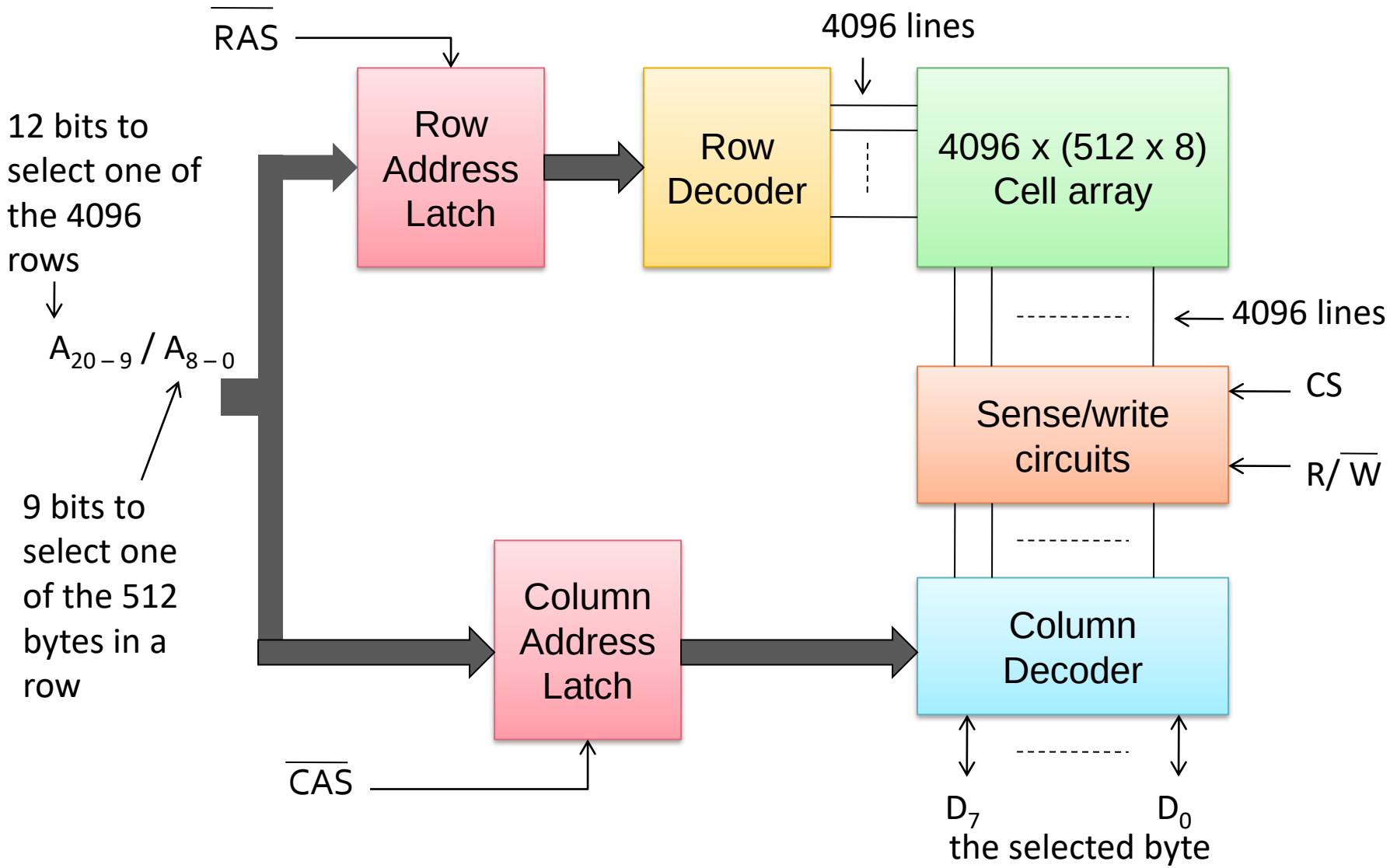


charge on the capacitor will discharge over time

A single transistor dynamic memory cell

- If a Read detects a voltage on the capacitor above the “threshold, ” it “sees” a 1, and drives the bit line to full voltage and recharges the capacitor
- If a Read detects a voltage on the capacitor below the “threshold, ” it “sees” a 0, and drives the bit line to ground and fully discharges the capacitor
- Refreshes whenever read Refresh circuit will periodically read all cells

Internal Organization of DRAM



Internal Organization of a 2M x 8 dynamic memory chip

DRAM

- Possible to leave row selected (all 512 bytes on sense lines)
- Then rapidly retrieve successive bytes by changing column addresses
- Result is a “fast page mode” for “blocks” or “pages” of bytes where appropriate (such as cache loading, disk transfer)
- Or, *synchronous* DRAM, SDRAM

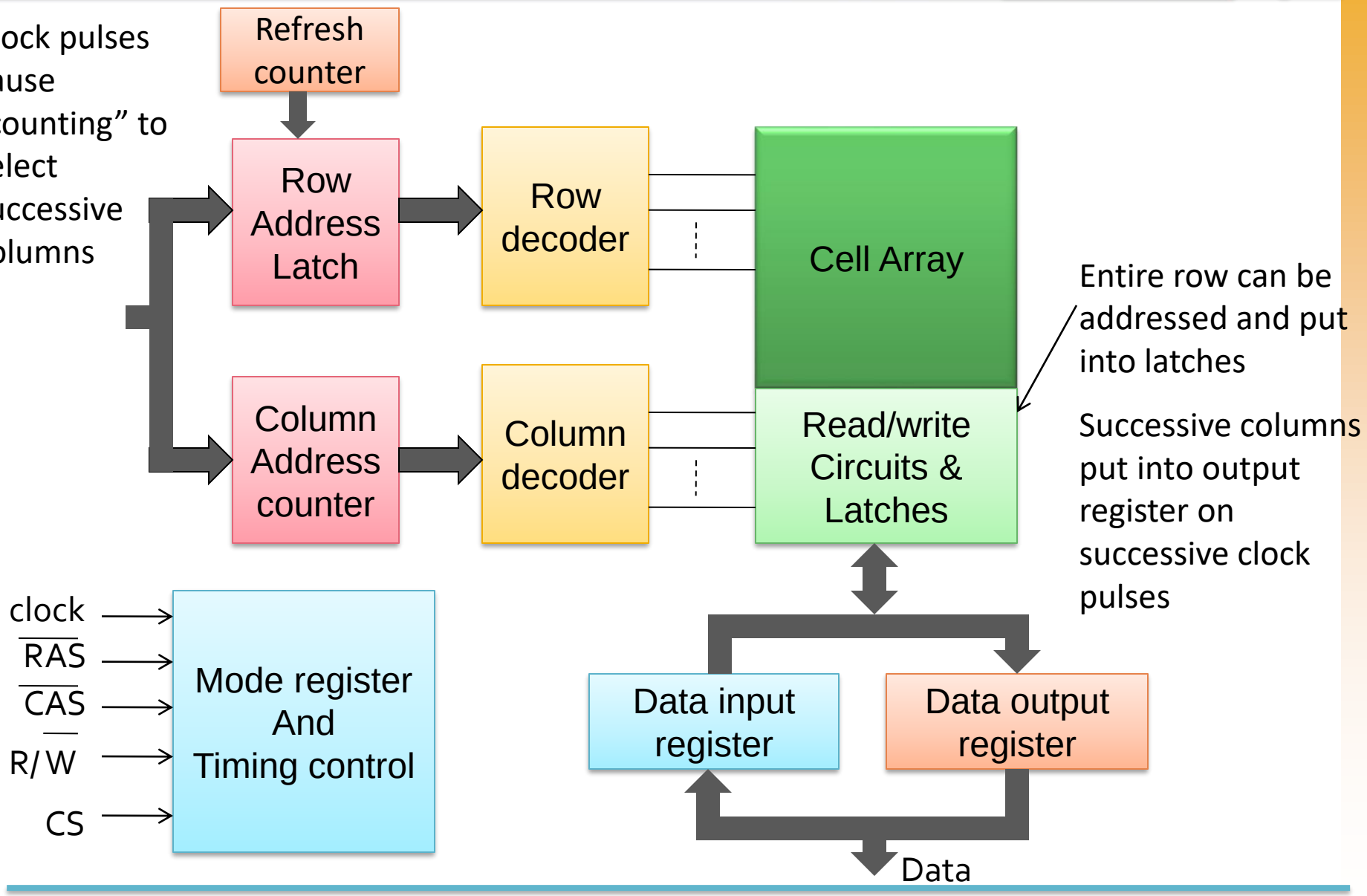
Latency and Bandwidth

- The speed and efficiency of data transfers among memory, processor, and disk have a large impact on the performance of a computer system.
- Memory latency – the amount of time it takes to transfer a word of data to or from the memory.
- Memory bandwidth – the number of bits or bytes that can be transferred in one second. It is used to measure how much time is needed to transfer an entire block of data.
- Bandwidth is not determined solely by memory. It is the product of the rate at which data are transferred (and accessed) and the width of the data bus.

DDR SDRAM

- Double-Data-Rate SDRAM
- Standard SDRAM performs all actions on the rising edge of the clock signal.
- DDR SDRAM accesses the cell array in the same way, but transfers the data on both edges of the clock.
- The cell array is organized in two banks. Each can be accessed separately.
- DDR SDRAMs and standard SDRAMs are most efficiently used in applications where block transfers are prevalent.

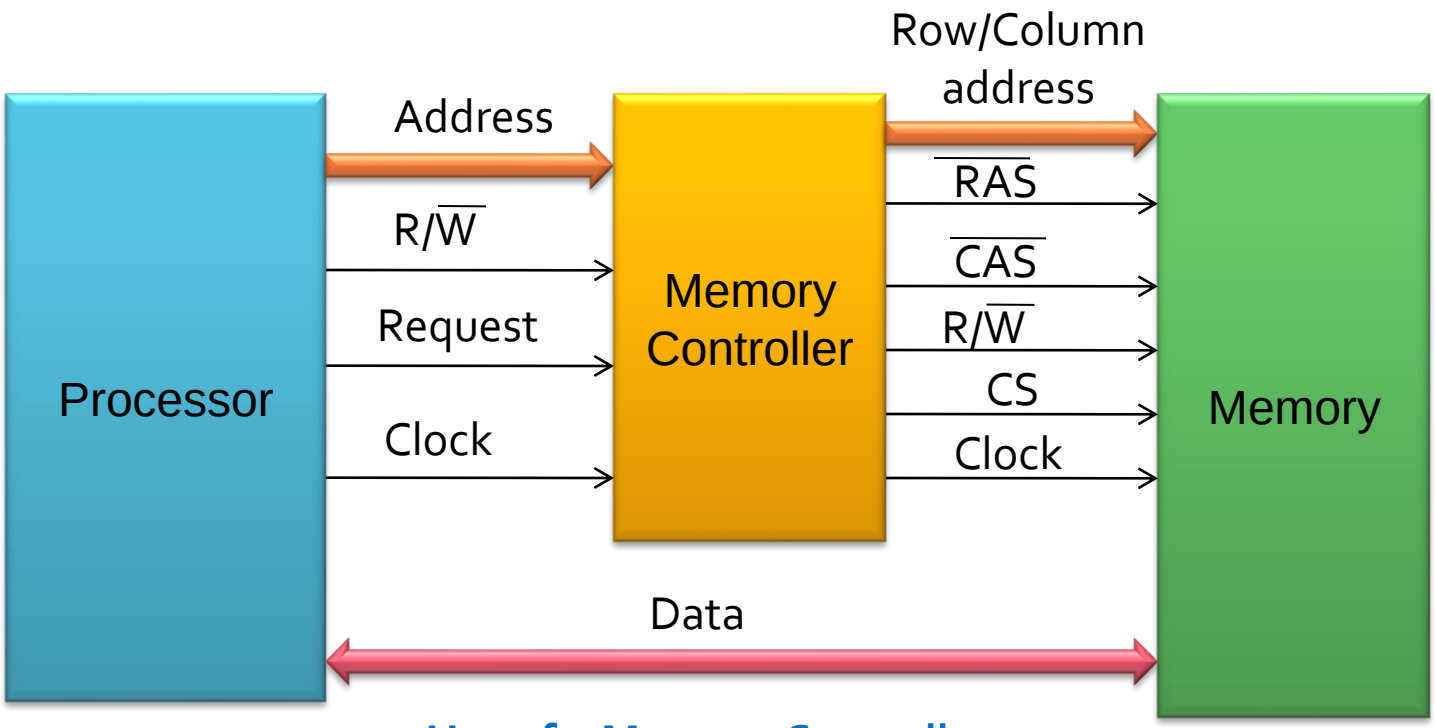
Synchronous DRAM



Memory System Considerations

- The choice of a RAM chip for a given application depends on several factors
  - Cost,
  - Speed
  - Power dissipation and
  - Size of the chip
- Static RAMs are generally used only when very fast operation is the primary requirement
  - They are used mostly in cache memories
- Dynamic RAMs are the predominant choice for implementing computer main memories

Memory Controller

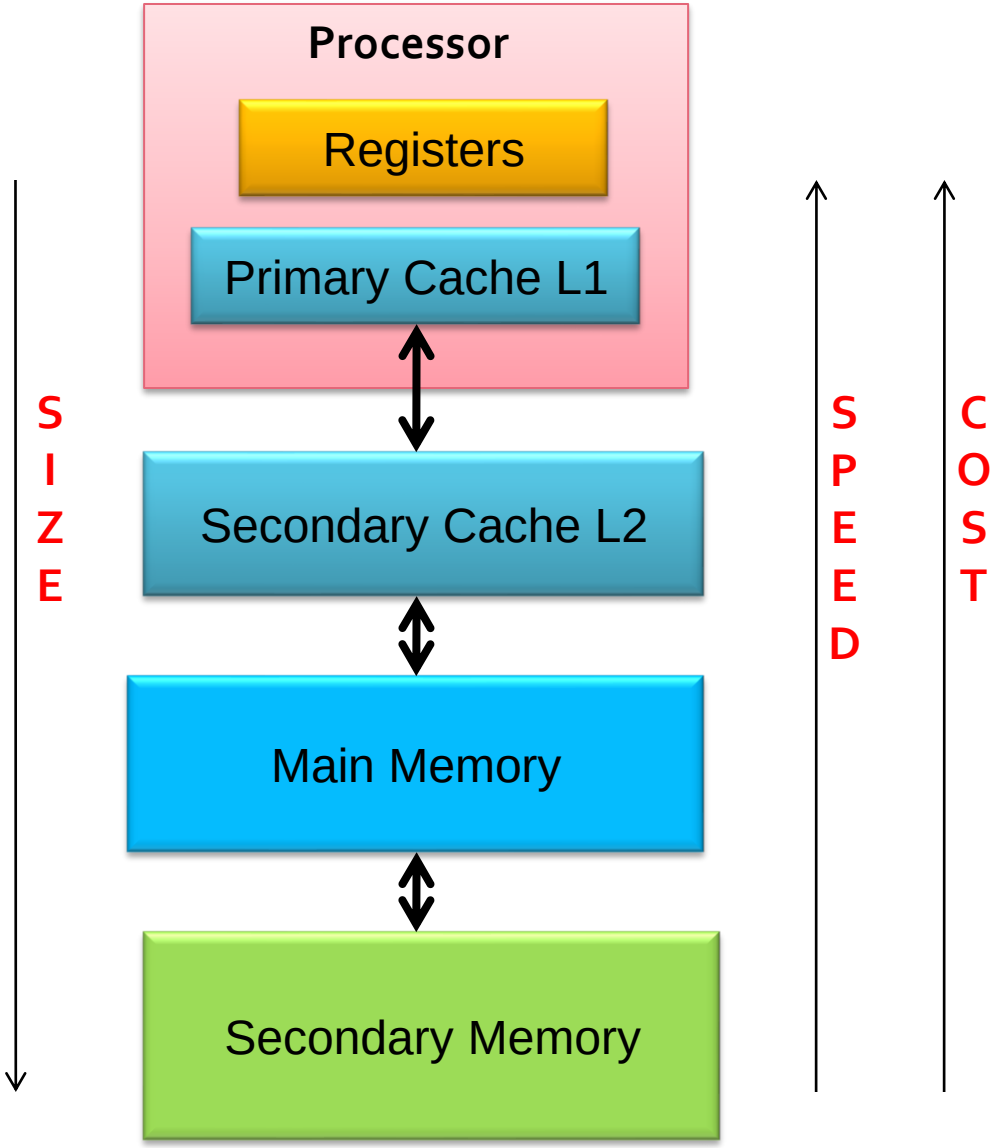


Use of a Memory Controller

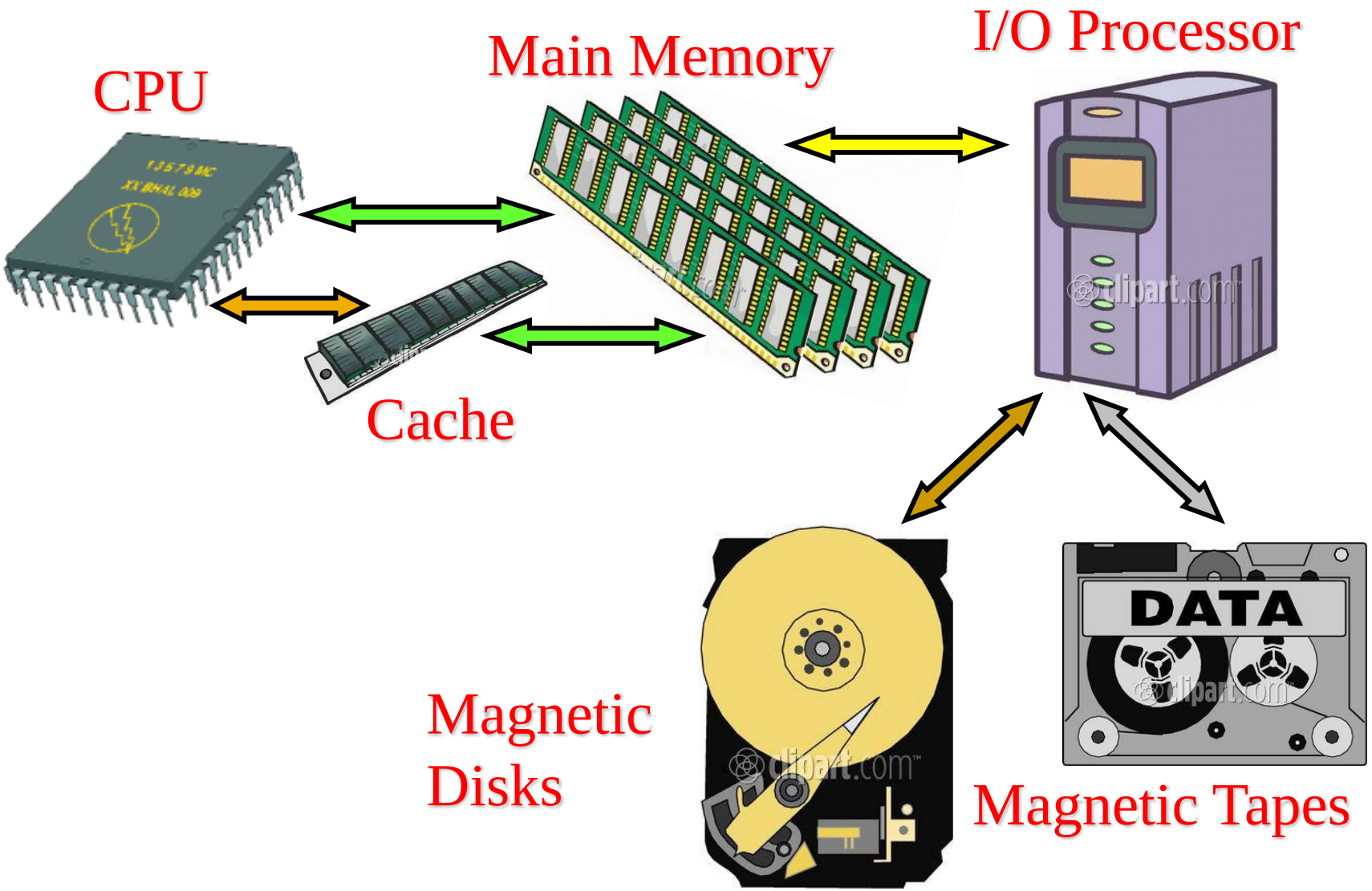
- Memory controller provides the refresh control if not done on the chip
  - Refreshing typically once every 64ms. At a cost of .2ms
  - Less than .4% overhead

Memory Hierarchy

- Memory hierarchy is to obtain the highest possible access speed
- while minimizing the total cost of the memory system



Memory Hierarchy



Cache Memories

- Main memory (still) slow in comparison to processor speed
- Main memory constrained by packaging, electronic characteristics and costs
- Cache memory on the processor chip typically ten times faster than main memory

Locality Reference

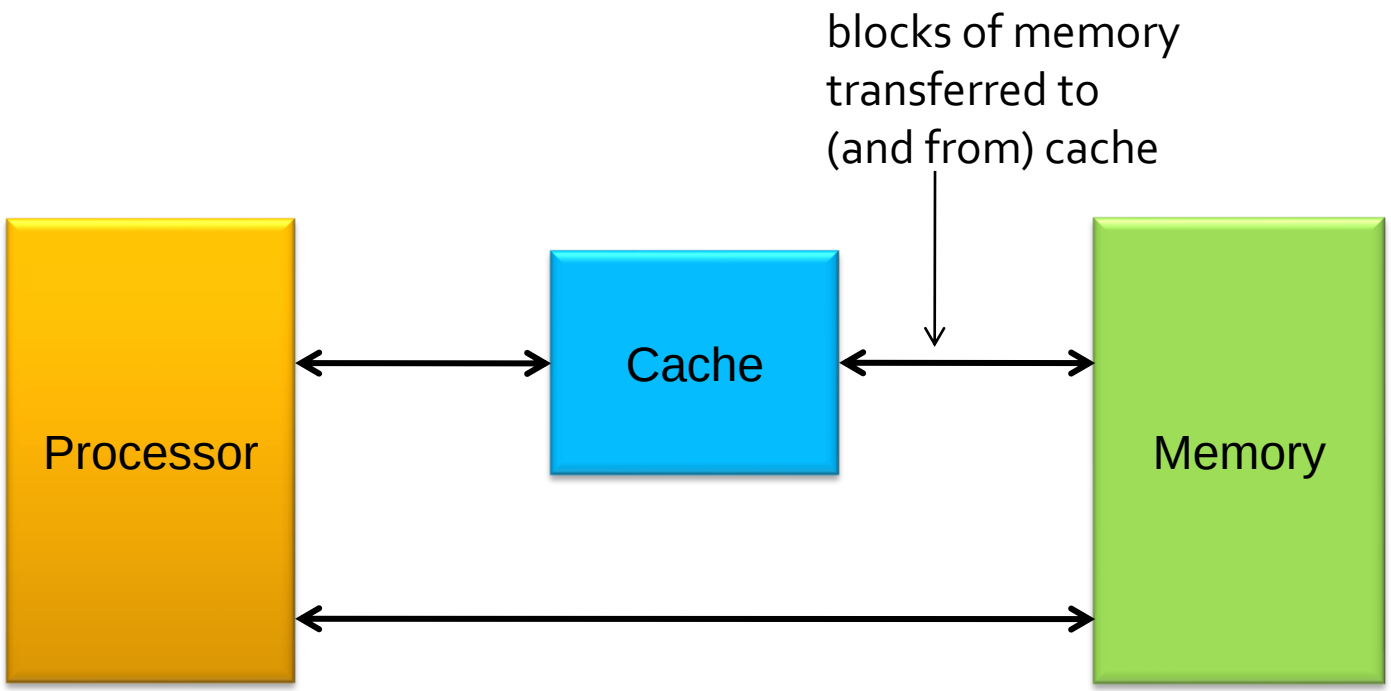
- Programs tend to spend their time “focused” on particular groups of instructions
  - Loops
  - Frequently called procedures

- “Localized” areas of programs executed repeatedly during some time period
- Much (most?) of program not accessed during some time period

Locality Reference

- Temporal
  - Recently executed instruction likely to repeat soon
  - When first accessed, move to cache where it will be when referenced again
- Spatial
  - Instructions near an executed instruction likely to be executed soon
  - When fetching an instruction from memory, move its neighbors into cache as well

Use of a Cache Memory

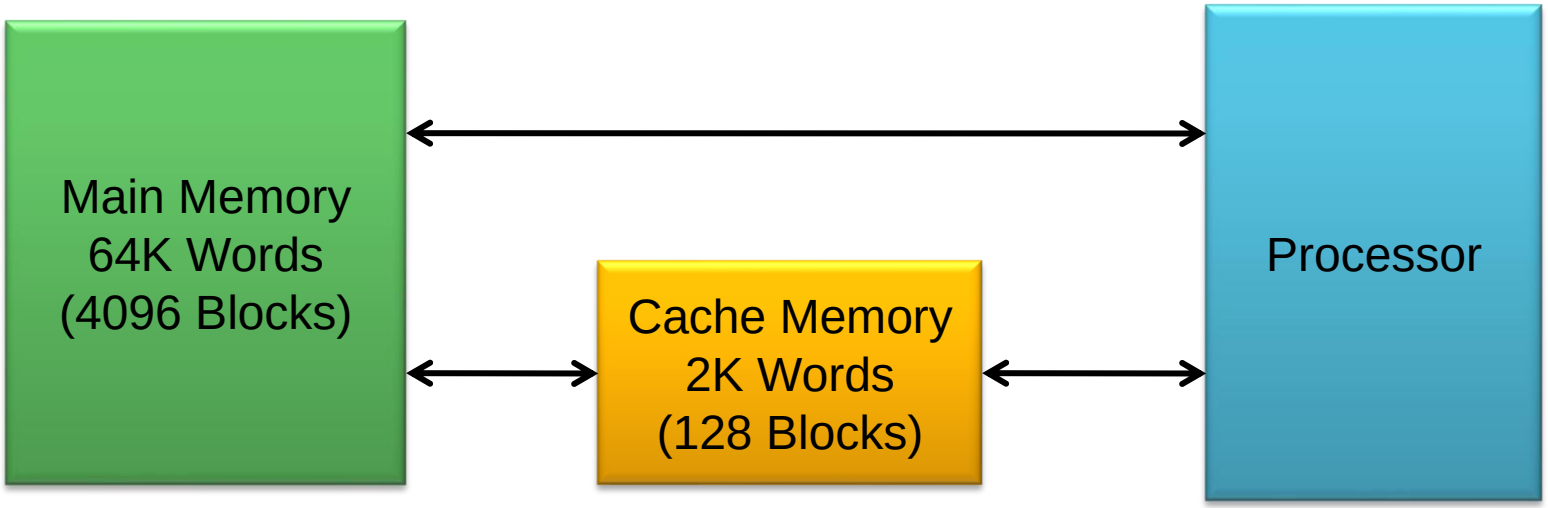


- Processor accesses instructions and data in the cache if there (a “hit”), in main memory if not (a “miss”)

Cache Memory

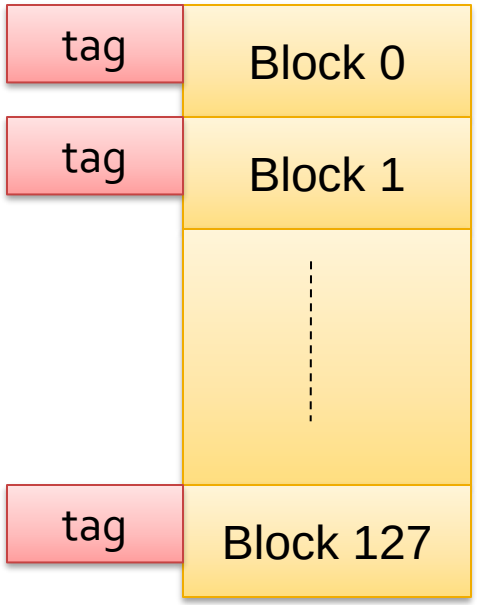
- Lets consider a **cache** consisting of 128 blocks of 16 words each, for total 2048 (2K)words
  - 128 x 16 words = 2048 words
  - Assume each block size is 16 words
- Lets consider the **main memory** is addressable by a 16-bit address.
  - 16 bit address ➔ 64 K words can be stored ➔ 4 K blocks can be stored (since each block size is 16 words)
- Main memory blocks can be modeled many way on to the cache
  - Cache is Content Addressable Memory (CAM)

Cache Memory

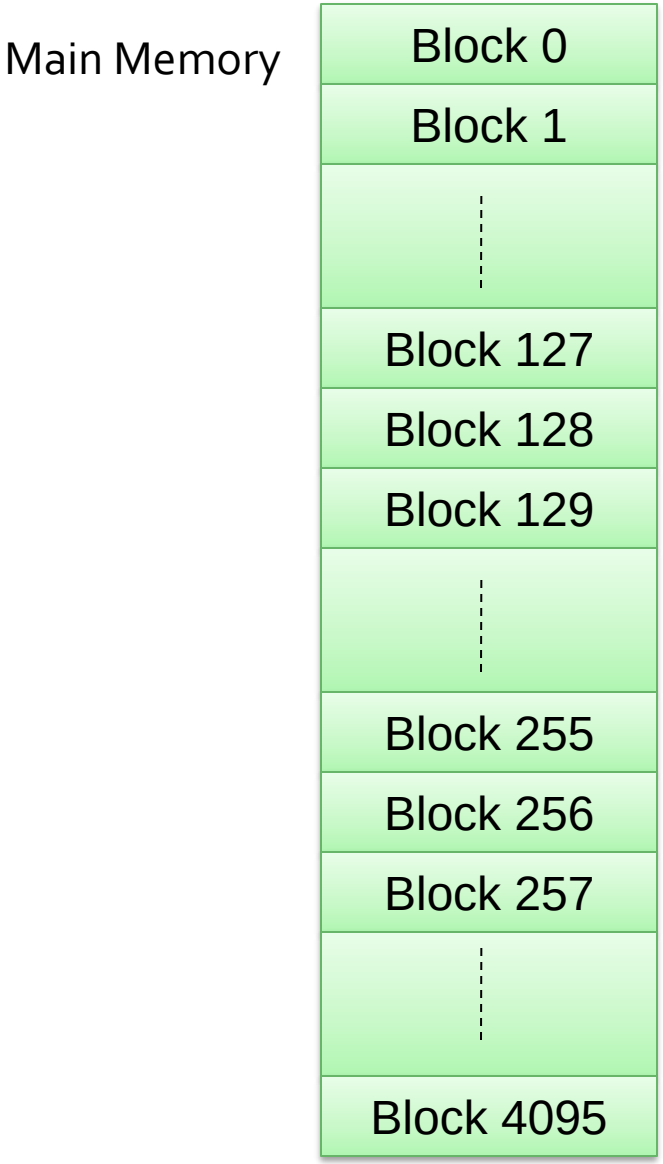


- Each block consists of 16 words

Cache and Main Memory



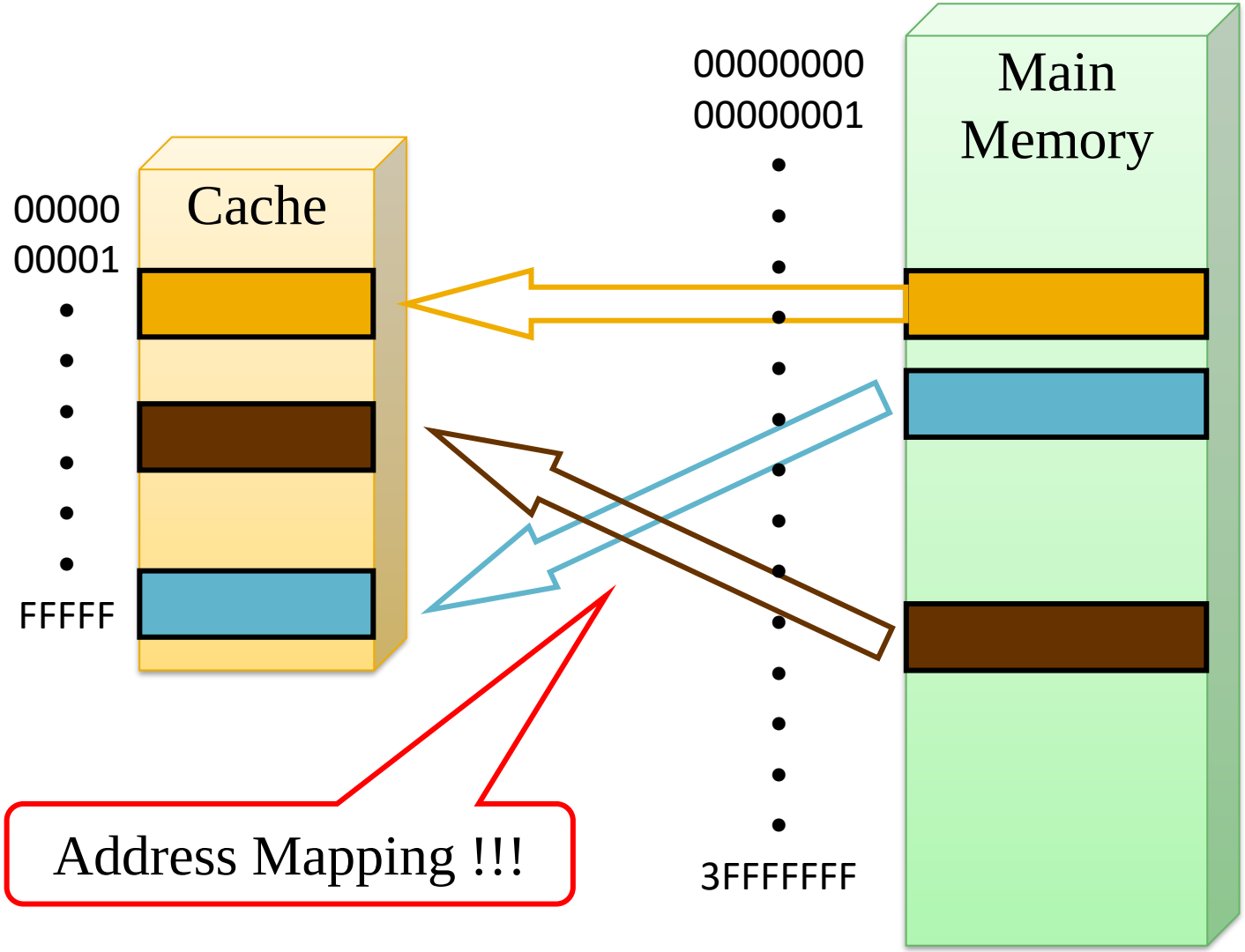
Cache Memory
2K Words
(128 Blocks)



Cache Management

- Mapping
  - Determination of where in the cache the blocks (cache lines) of main memory are to be placed
- Replacement
  - Determination of when to replace a block in cache with another block of main memory
- Coherency
  - Assurance that no problems arise from cache version differing from main memory version

Cache Mapping



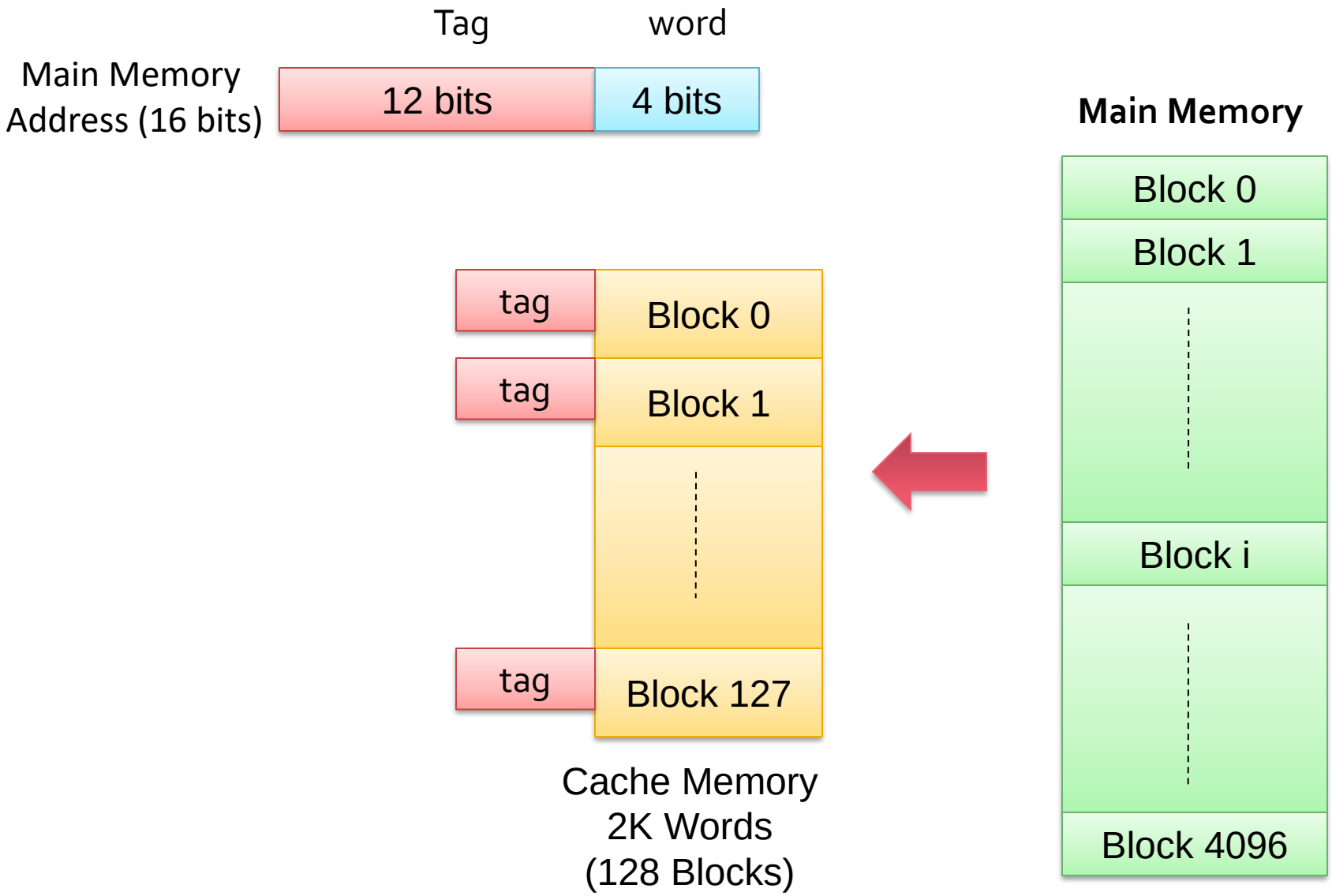
Memory and Cache mapping

- Mapping function
  - Specification of correspondence between main memory blocks and cache blocks
    - Associative mapping
    - Direct Mapping
    - Set- associative mapping

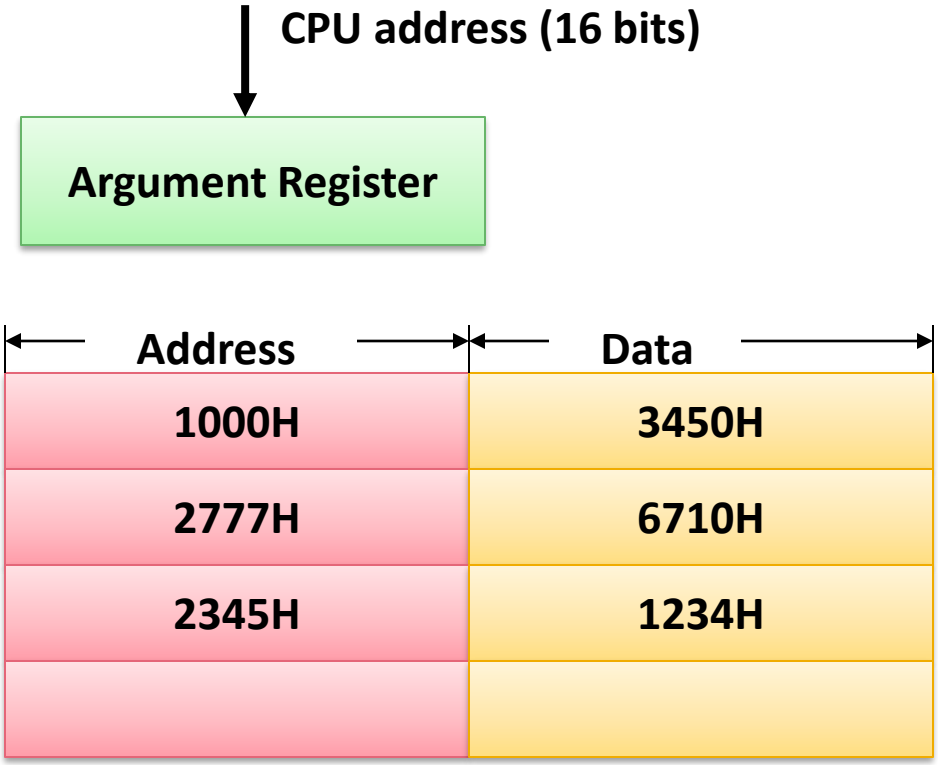
Associative Mapping

- Any block location in cache can store any block in memory
  - Most flexible
- Mapping table is implemented in an associative memory
  - Fast, very expensive
- Mapping table stores both address and the content of the memory word

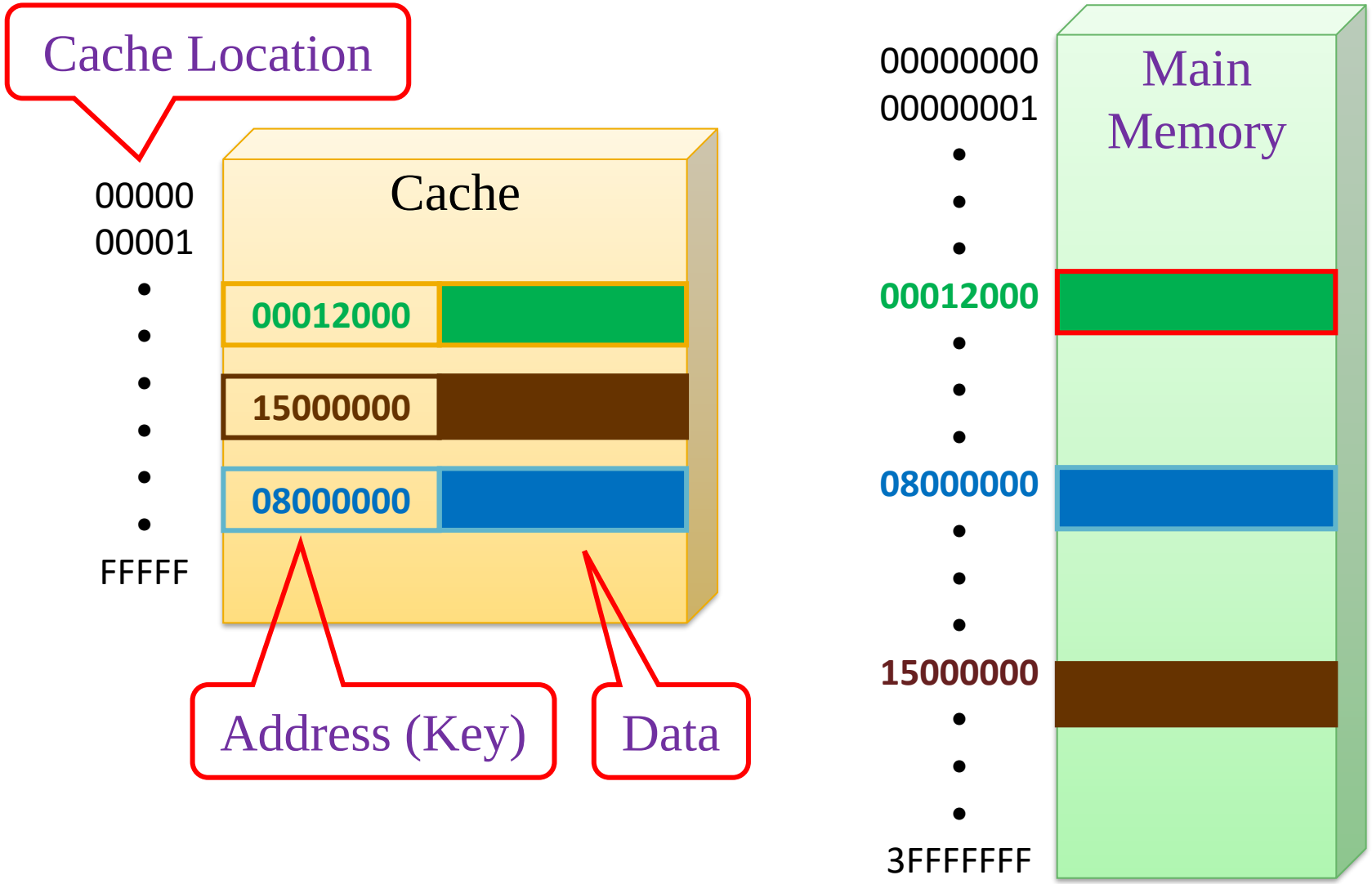
Associative Mapping



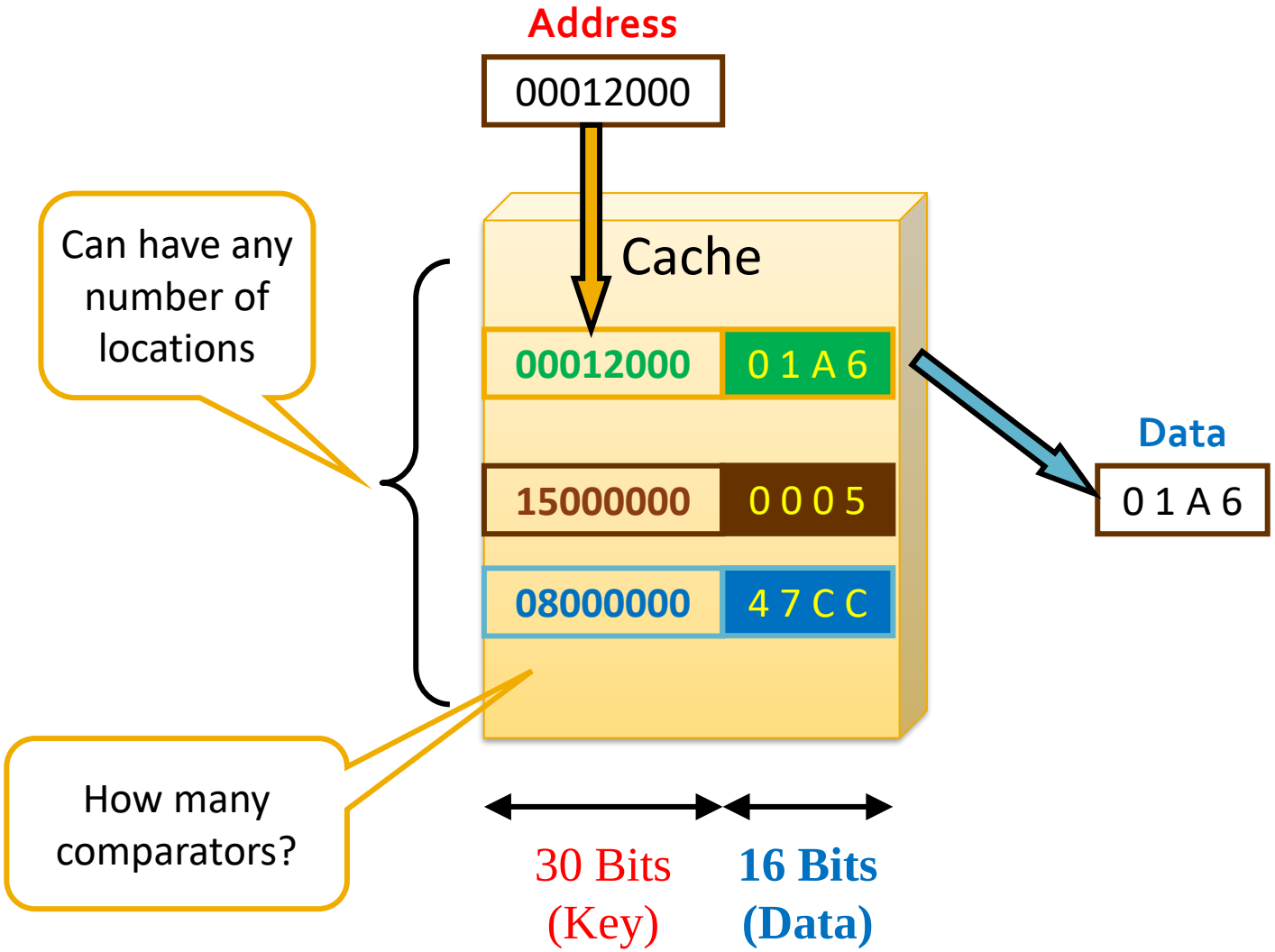
Associate Mapping Example



Associative Mapping



Associative Mapping



Associative Mapping

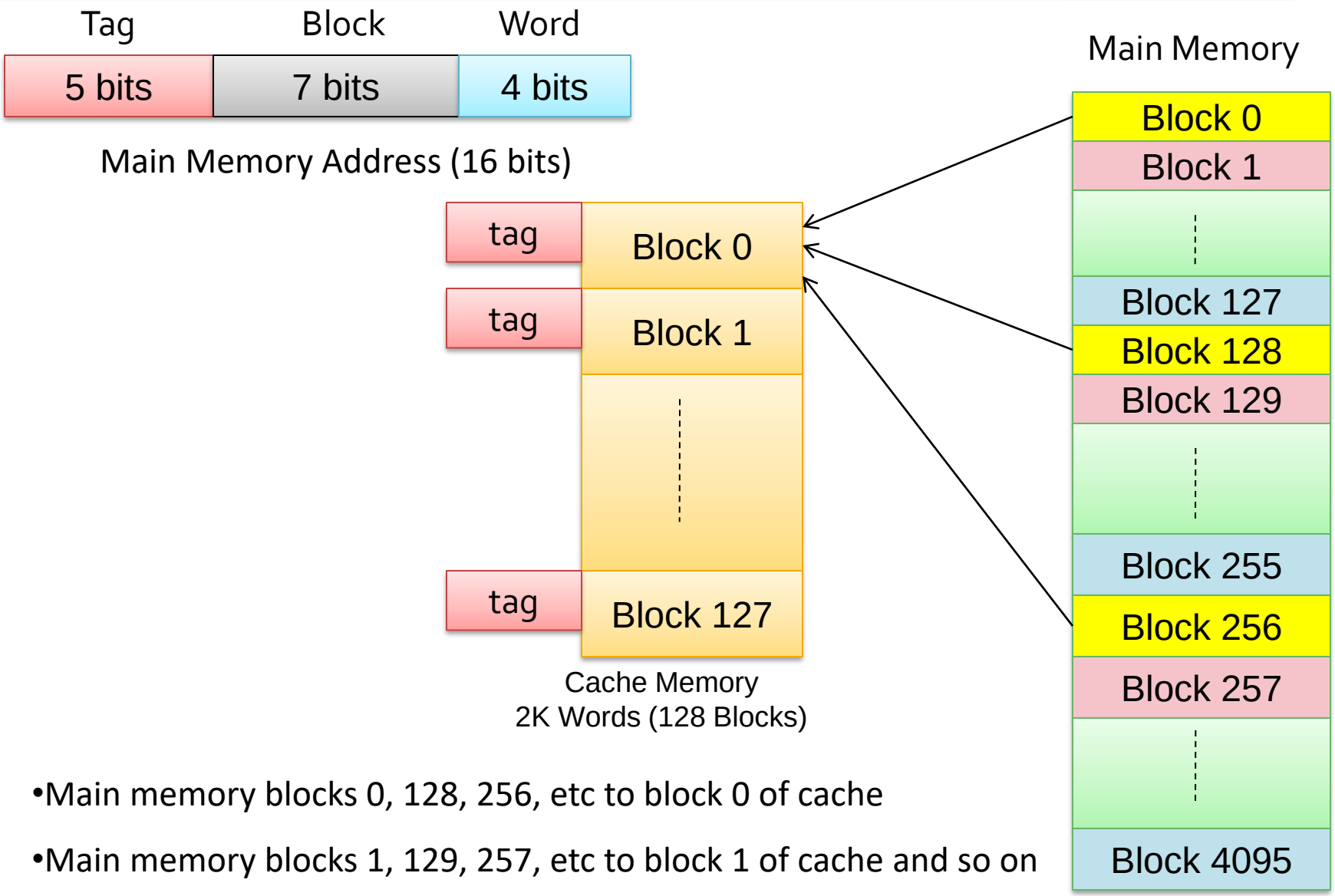
Tag	Word	
12	4	Main Memory Address
1110 1111 1111	1100	

- Tag: 111011111111
- Word:1100=12, the 12th word of a block in the cache

Direct Mapping

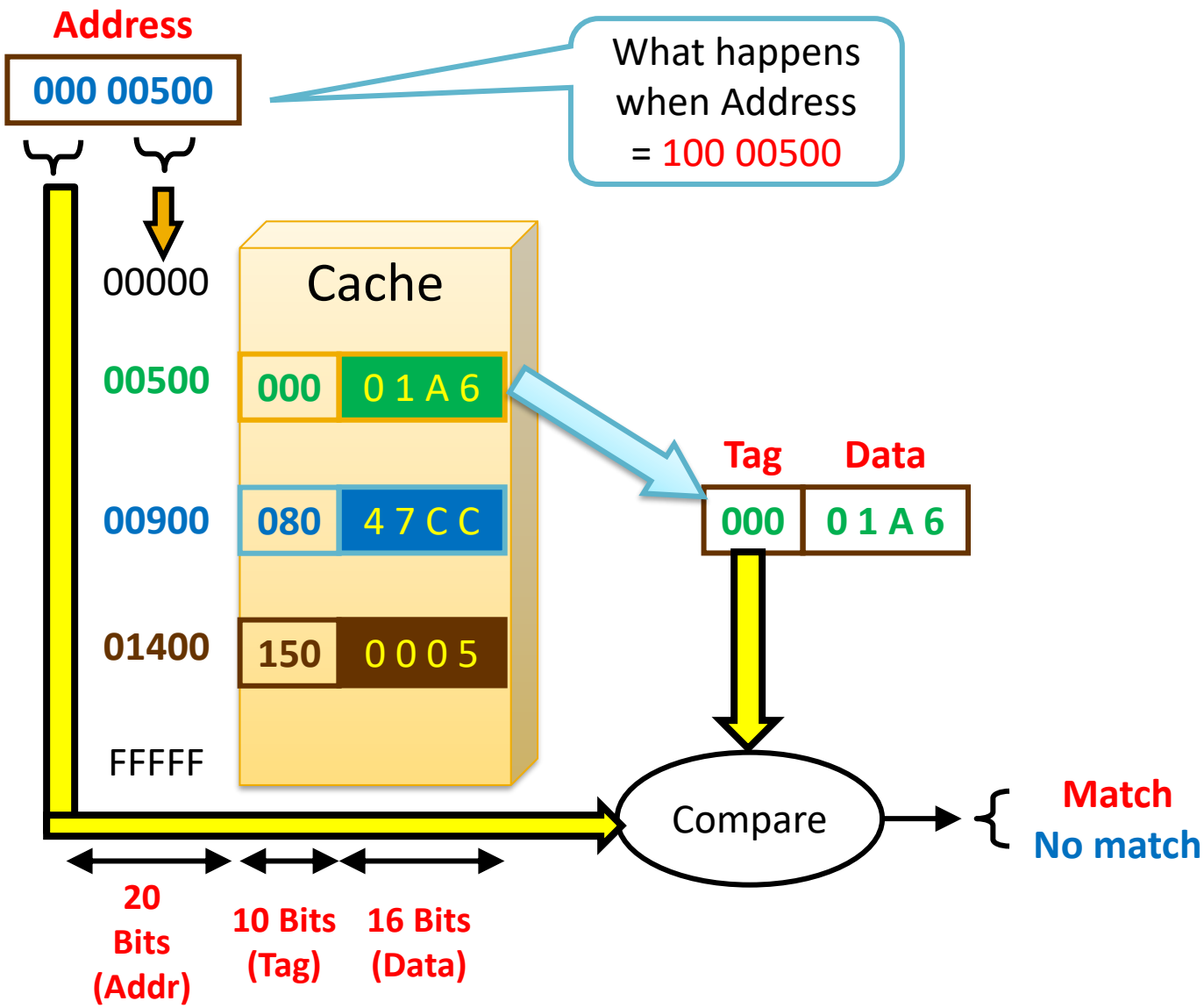
- Each memory block has only one place to load in Cache
- Mapping table made of RAM instead of CAM
- n-bit memory address consists of 2 parts
  - k-bits of index field
  - n-k bits of Tag field
- n-bit addresses are used to access main memory and k-bit index is used to access the cache

Direct Mapping

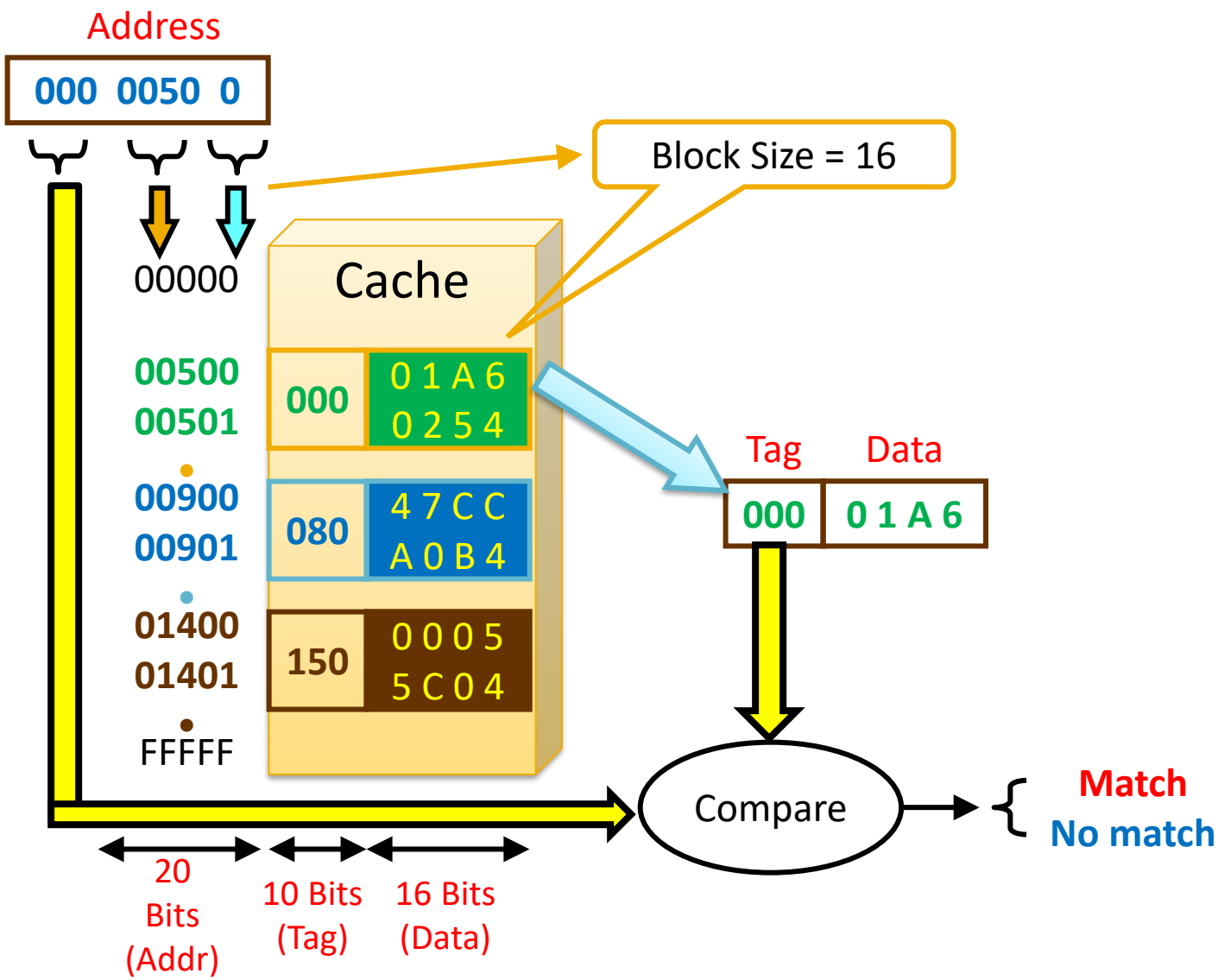


- Main memory blocks 0, 128, 256, etc to block 0 of cache
- Main memory blocks 1, 129, 257, etc to block 1 of cache and so on

Direct Mapping



Direct Mapping with Blocks



Direct Mapping

Tag	Block	Word	
5	7	4	Main Memory Address
11101	1111111	1100	

- Tag: 11101
- Block: 1111111=127, in the 127th block of the cache
- Word:1100=12, the 12th word of the 127th block in the cache

Direct Mapped Cache Example

* Address in octal form

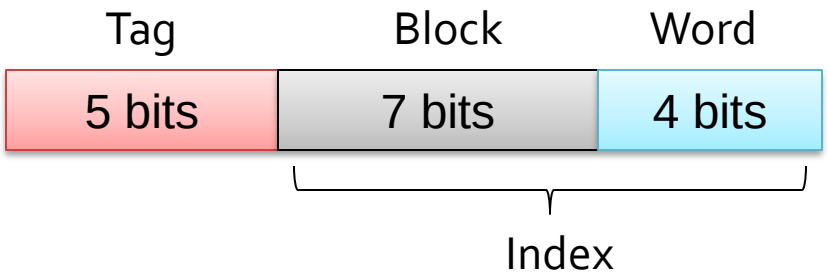
Memory Address	Memory Data
00000	1220
00777	2340
01000	3450
01777	4560
02000	5670
02777	6710

Main Memory

Index Address	Tag	Data
000	00	1220
777	02	6710

Cache Memory

Direct Mapping with Block Example



	Index	Tag	Data
Block 0	000	01	3450
	00F	01	6578
Block 1	010		
	01F		
Block 127	7F0	02	
	7FF	02	6710

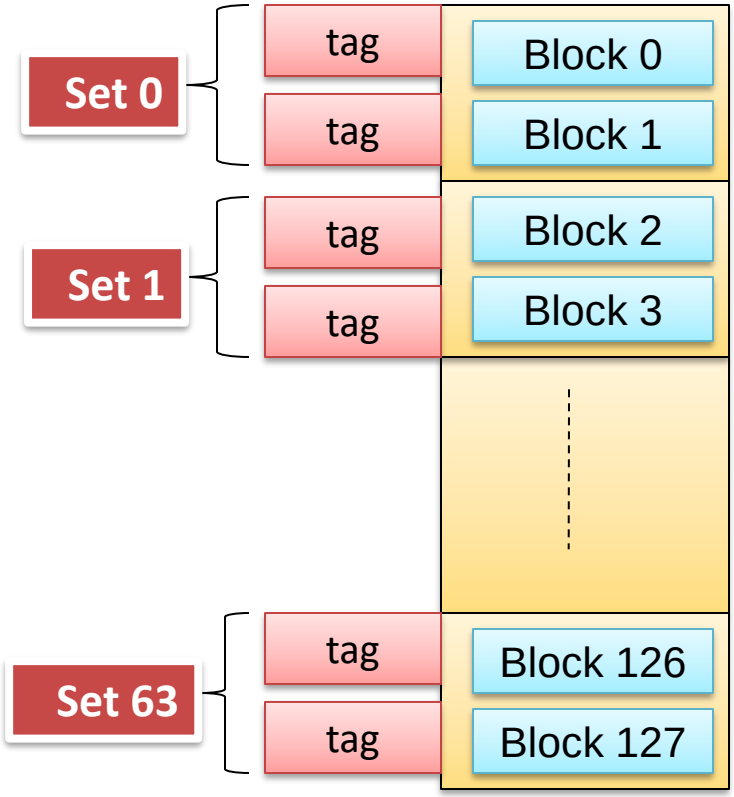
Set Associative Mapping

- A combination of direct and associative mapping techniques can be used
- Blocks of the cache are grouped into sets and the mapping allows a block of the main memory to reside in any block of a specific set

Set Associative Mapping

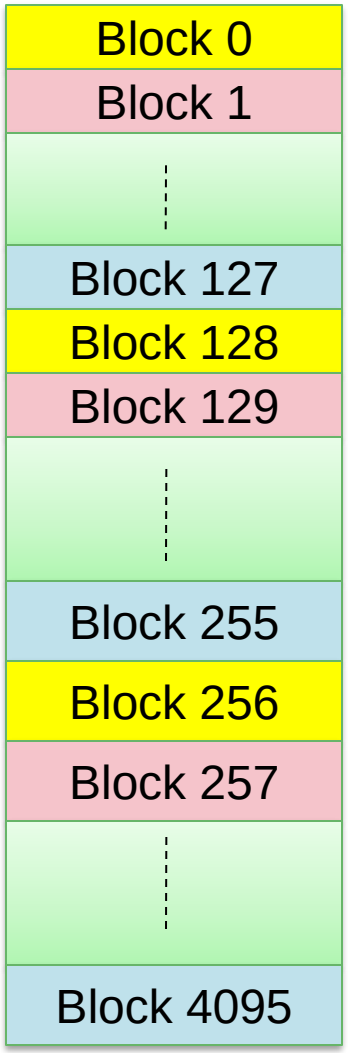
Tag	Block	Word
6 bits	6 bits	4 bits

Main Memory Address (16 bits)



Cache Memory 2K Words (128 Blocks)

Main Memory

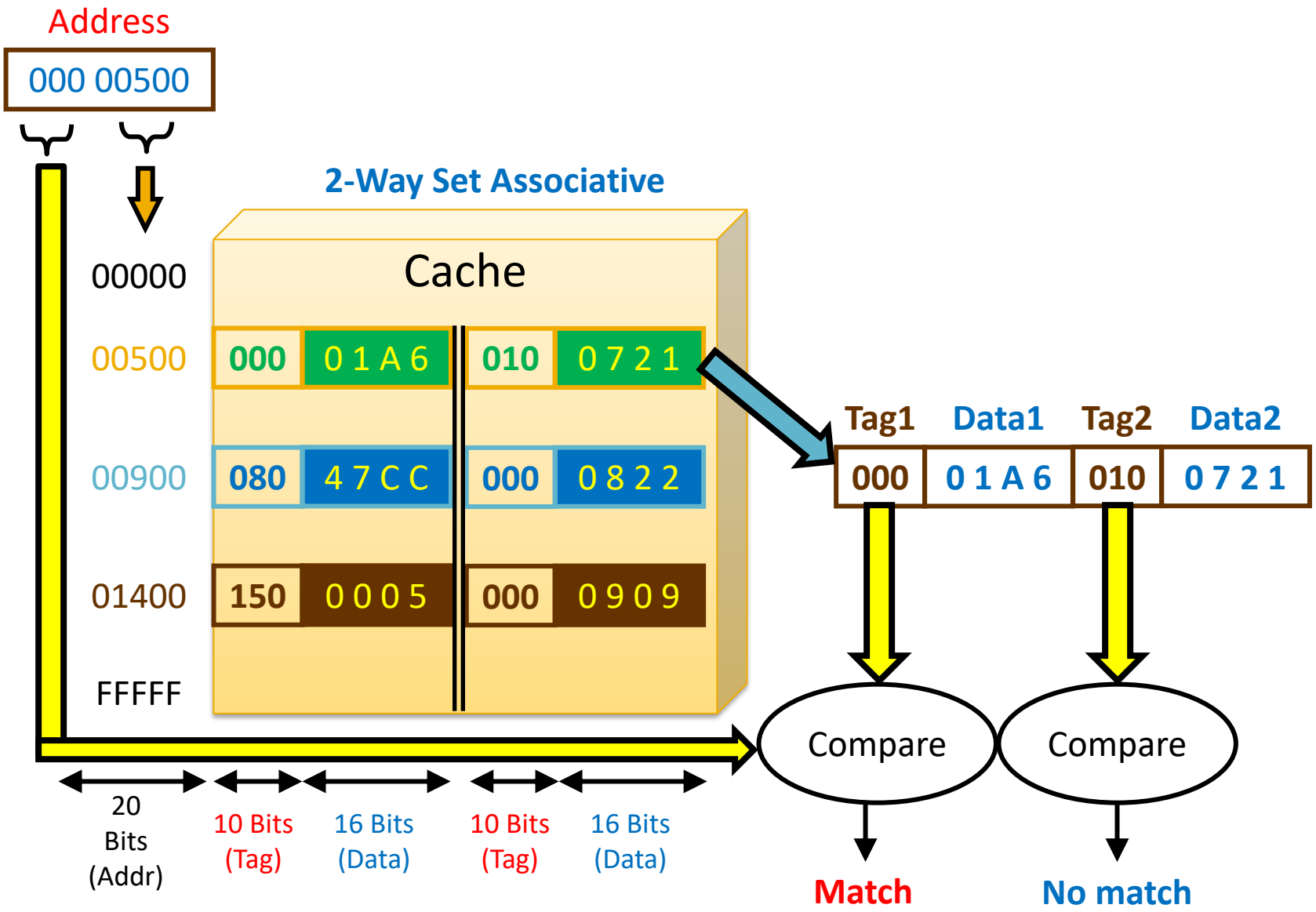


Set Associative Mapping Example

- Each memory block has a set of locations in the cache to load
- Set associative mapping cache with set size of two

Index	Tag	Data	Tag	Data
000	01	3450	02	5670
777	02	6710	00	2340

Set-Associative Mapping



Set-Associative Mapping

Tag	Block	Word
6 bits	6 bits	4 bits

Main Memory Address (16 bits)

1 1 1 0 1 1	1 1 1 1 1 1	1 1 0 0

- Tag: 111011
- Set: 111111=63, in the 63th set of the cache
- Word:1100=12, the 12th word of the 63th set in the cache

Replacement Algorithms

- Difficult to determine which blocks to kick out
- Least Recently Used (LRU) block
- The cache controller tracks references to all blocks as computation proceeds
- Increase / clear track counters when a hit/miss occurs

Replacement Algorithms

- For Associative & Set-Associative Cache
  - Which location should be emptied when the cache is full and a miss occurs?
  - First In First Out (FIFO)
  - Least Recently Used (LRU)
- Distinguish an *Empty* location from a *Full* one
  - Valid Bit

Cache Write

- Write Through
  - When writing into memory
    - If hit both cache and Main memory is written in parallel
    - If Miss, Memory is written
      - For a read miss, missing block may be overloaded onto a cache block
  - Memory is always updated
    - Important when CPU and DMA I/O are both executing
    - Slow – due to the memory access time

Cache Write

- Write Back (Update Cache only)
  - When writing into Memory
    - If hit, only cache is written
    - If miss, missing block is brought to cache and write into cache
    - For a read miss, candidate block must be written back to the memory
  - Memory is not up-to-dated, i.e. the same item in cache and memory may have different value
  - Mark cache block “dirty” or “modified”
  - Copy it back to main memory when another block needs the cache space

Performance Considerations

Overview

- Two key factors: performance and cost
- Price/performance ratio
- Performance depends on how fast machine instructions can be brought into the processor for execution and how fast they can be executed.
- For memory hierarchy, it is beneficial if transfers to and from the faster units can be done at a rate equal to that of the faster unit.
- This is not possible if both the slow and the fast units are accessed in the same manner.
  - However, it can be achieved when parallelism is used in the organizations of the slower unit (interleaved Organization).

Hit Rate and Miss Penalty

- The success rate in accessing information at various levels of the memory hierarchy – hit rate / miss rate.
- Ideally, the entire memory hierarchy would appear to the processor as a single memory unit that has the access time of a cache on the processor chip and the size of a magnetic disk – depends on the hit rate (>>0.9).
- A miss causes extra time needed to bring the desired information into the cache.

Hit Rate and Miss Penalty (cont.)

- $T_{ave} = hC + (1-h)\,M$
  - $T_{ave:}$ average access time experienced by the processor
  - h: hit rate
  - M: miss penalty, the time to access information in the main memory
  - C: the time to access information in the cache

How to Improve Hit Rate?

- Use larger cache – increased cost
- Increase the block size while keeping the total cache size constant.
- However, if the block size is too large, some items may not be referenced before the block is replaced – miss penalty increases.

Caches on the Processor Chip

- On chip vs. off chip
- Two separate caches for instructions and data, respectively
- Single cache for both
- Which one has better hit rate? -- Single cache
- What's the advantage of separating caches? – parallelism, better performance
- Level 1 and Level 2 caches
- L1 cache – faster and smaller. Access more than one word simultaneously and let the processor use them one at a time.
- L2 cache – slower and larger.
- How about the average access time?
- Average access time: $t_{ave} = h_1C_1 + (1-h_1)h_2C_2 + (1-h_1)(1-h_2)M$ where h is the hit rate, C is the time to access information in cache, M is the time to access information in main memory.

Performance of cache

- Memory Access
  - All the memory accesses are directed first to cache
  - If the word is in the cache; access cache provide it to CPU
  - If the word is not in cache; bring a block including that word to replace a block now in cache

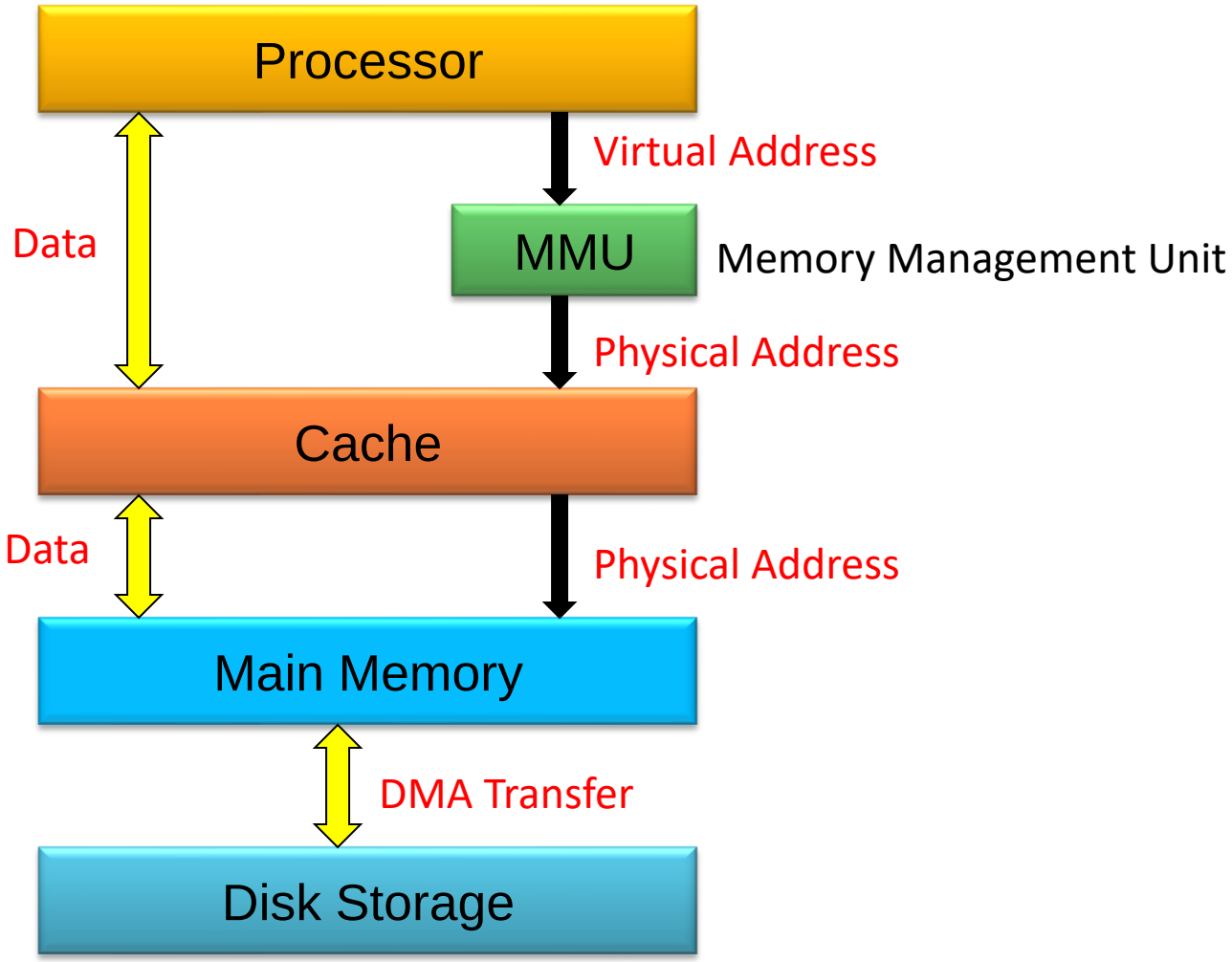
Virtual Memories

Introduction

- Physical main memory is not as large as the address space spanned by an address issued by the processor.

  2^{32} = 4 GB, 2^{64} = …
- When a program does not completely fit into the main memory, the parts of it not currently being executed are stored on secondary storage devices.
- Techniques that automatically move program and data blocks into the physical main memory when they are required for execution are called virtual-memory techniques.
- Virtual addresses will be translated into physical addresses.

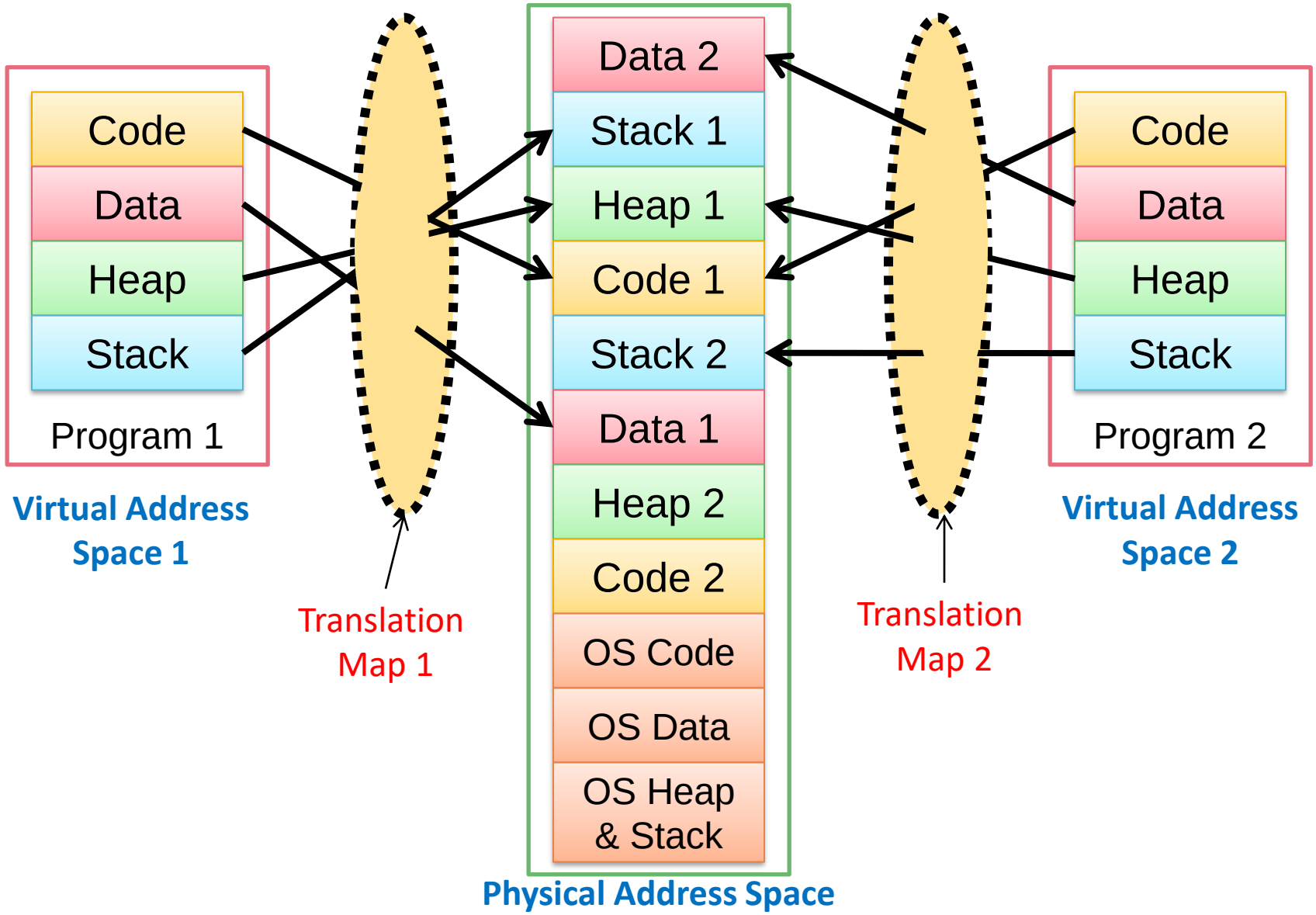
Virtual Memory Organization



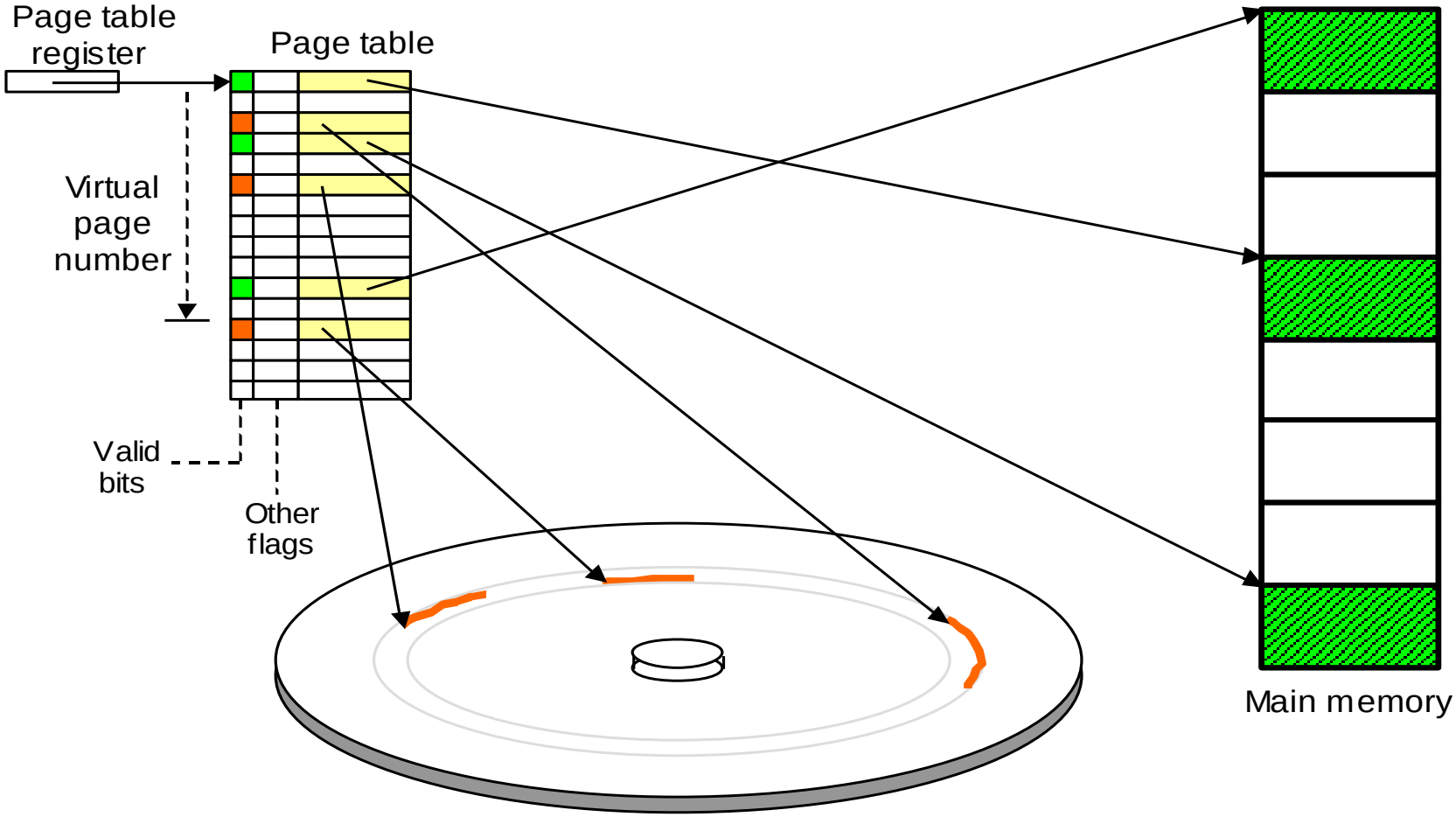
Address Translation

- All programs and data are composed of fixed-length units called pages
  - each of which consists of a block of words that occupy contiguous locations in the main memory.
- Page cannot be too small or too large.
- The virtual memory mechanism bridges the size and speed gaps between the main memory and secondary storage
  - similar to cache.

Example of Address Translation

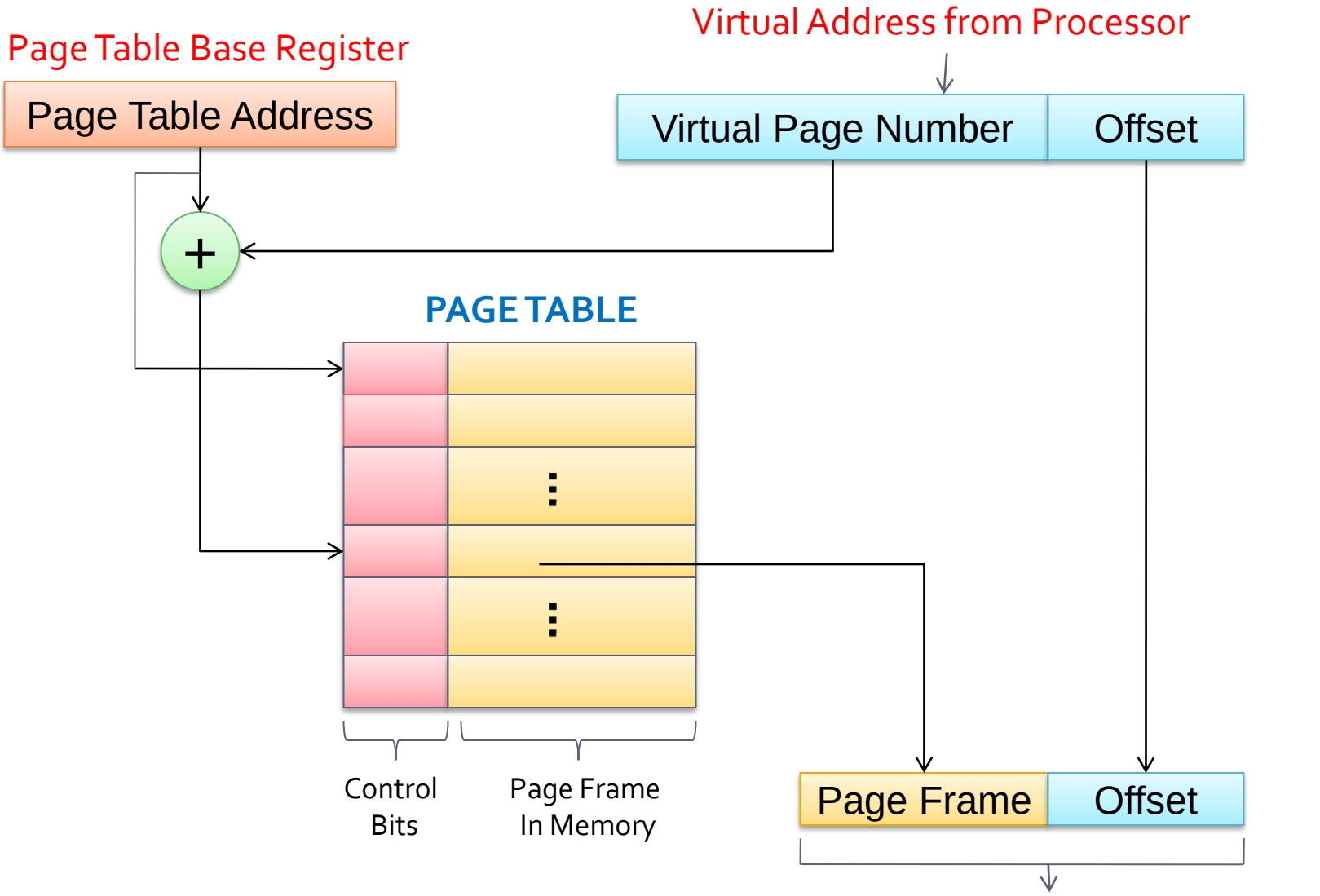


Page Tables and Address Translation



The role of page table in the virtual-to-physical address translation process.

Address Translation



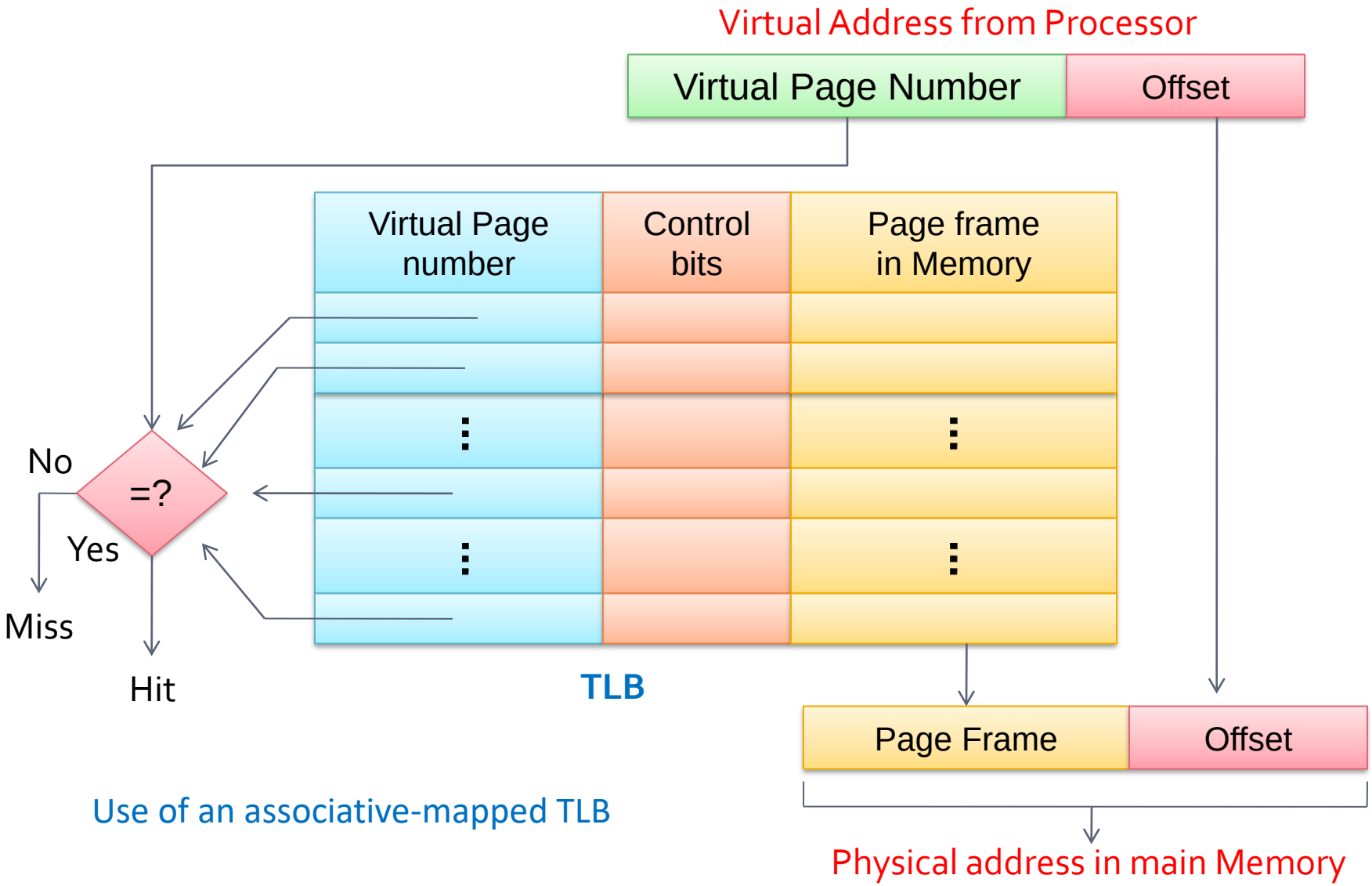
Address Translation

- The page table information is used by the MMU for every access, so it is supposed to be with the MMU.
- However, since MMU is on the processor chip and the page table is rather large, only small portion of it, which consists of the page table entries that correspond to the most recently accessed pages, can be accommodated within the MMU
- It is impossible to include a complete page table on this chip
- Therefore, page table is kept in the Main Memory
  - A copy of a small portion of the page table can be accommodated within the MMU

TLB

- This portion is consists of the page table entries that correspond to the most recently accessed pages
  - A small cache, usually called the Translation Lookaside Buffer (TLB) incorporated into the MMU for this purpose

Translation Lookaside Buffer (TLB)



Use of an associative-mapped TLB

Translation Lookaside Buffer (TLB)

- The contents of TLB must be coherent with the contents of page tables in the memory
- Translation procedure
- Page fault
  - An access request to the page that is not there in the Main Memory
- Page replacement
- Write-through is not suitable for virtual memory
- Locality of reference in virtual memory

Memory Management Requirements

- Multiple programs
- System space / user space
- Protection (supervisor / user state, privileged instructions)
- Shared pages

Thank You!!